

Universidad de Ingeniería y Tecnología



Proyecto I

Implementación de un sistema de gestión de bases de datos

Base de Datos II (CS2042)

Ciclo 2026-2

Integrantes

Anyeli Azumi Tamara Ureta
Carlos Alberto Villegas Arce
Héctor Junior Caballero Canchanya
Irán María Alvarez Flores
Kenzo Akira Shibuya Rengifo

Docente

Heider Sánchez

Lima, Perú
5 de octubre de 2026

Índice

1. Introducción	1
2. Arquitectura del sistema	1
2.1 Capas y módulos	1
2.2 Flujo de una consulta	3
3. Dominio de datos	3
4. Gestión de almacenamiento	4
4.1 FileManager	4
4.2 Buffer pool	5
4.3 Segment	6
4.4 Slotted page, registro y RID	7
5. Organización de archivos	8
5.1 Heap File	8
5.2 Sequential File	9
5.3 Comparación	12
6. Estructuras de indexación	13
6.1 Codificación de claves y páginas de nodo	13
6.2 B+ Tree no agrupado	14
6.3 B+ Tree agrupado	16
6.4 Hash extendible	17
6.5 R-Tree	18
7. Algoritmos externos	25
7.1 External sort	25
7.2 External hash	26
8. Procesamiento de consultas SQL	28
8.1 Lexer, parser y AST	28
8.2 Análisis semántico	30
8.3 Catálogo	31
8.4 Planificación y ejecución	31
8.5 Extensión espacial	33
8.6 Plan de ejecución (EXPLAIN)	35
9. Transacciones y concurrencia	36
9.1 Lock manager	36
9.2 Transaction manager	37
9.3 Serializabilidad	38
9.4 Simulación con hilos	38
9.5 Resultados de la simulación	39

10.API e interfaz de usuario	40
10.1 API REST	40
10.2 Interfaz	41
10.3 Inspector físico	42
10.4 Laboratorio espacial	43
10.5 Importación CSV	44
11.Comparación experimental	44
11.1 Entorno y metodología	44
11.2 Heap File frente a Sequential File	45
11.3 B+ agrupado, B+ no agrupado y hash extendible	47
11.4 Búsqueda secuencial, R-Tree y GiST de PostgreSQL	52
11.5 Cuándo usar cada técnica	55
12.Conclusiones	57

Índice de cuadros

1	Tablas de demostración que crea el backend.	4
2	Costos en I/O de página.	12
3	Cobertura SQL implementada.	28
4	Matriz de compatibilidad.	36
5	Escenario de carrera con 8 hilos y 3 claves en disputa.	39
6	Contrato REST implementado.	40
7	Diseño del benchmark espacial.	45
8	Heap File frente a Sequential File; una ejecución, 30 búsquedas.	46
9	Tiempo de reorganización del Sequential File con registros borrados.	47
10	Construcción y espacio de los índices relacionales.	48
11	Tiempo promedio de consulta en ms. Todas las mediciones devuelven las filas completas.	48
12	Igualdad sobre carrera , que tiene 8 valores, en ms.	49
13	Rendimiento de los índices relacionales ante mutaciones frecuentes ($N =$ 10 000, 500 inserciones y 500 eliminaciones intercaladas).	51
14	Construcción, espacio en disco y memoria de las técnicas espaciales.	53
15	Tiempo promedio de 100 consultas espaciales en ms.	54
16	Consultas que resuelve cada índice relacional.	55
17	Comparación de los índices relacionales.	55
18	Comparación de las organizaciones de archivo.	56
19	Comparación de las técnicas espaciales.	57

Índice de figuras

1	Arquitectura por capas del minigestor. El ejecutor consulta el catálogo, selecciona algoritmos externos, archivos o índices, y delega el acceso físico al buffer pool y al FileManager.	2
2	Recorrido de una sentencia SQL por los módulos del sistema.	3
3	Capas de almacenamiento compartidas por todas las estructuras.	5
4	Reemplazo LRU en un buffer pool de cinco frames.	6
5	Free list de un archivo de índice y orden en que se asignan las páginas.	7
6	Slotted page de un archivo de datos.	7
7	Compactación de una slotted page. El hueco que dejó el registro borrado de s2 se junta con el espacio libre.	8
8	Inserción en Heap File con reutilización de espacio.	9
9	Lista enlazada del Sequential File en orden de clave, a través de MAIN y AUX.	9
10	Inserción en el Sequential File y verificación de reorganización.	11
11	Reorganización del Sequential File con 3 registros por página.	12
12	Búsqueda por igualdad en I/O de página, con AUX en su tamaño máximo.	13
13	Páginas de 4 KB del B+ Tree y del hash extendible.	15
14	B+ Tree no agrupado sobre una tabla Heap.	15

15	Split de una hoja llena del B+ Tree al insertar 35.	16
16	B+ agrupado disperso, con una entrada por página de MAIN.	17
17	Hash extendible con global depth 2 y capacidad 4 por bucket.	18
18	Inserción en el hash extendible.	19
19	Páginas de 4 KB de una hoja y de un nodo interno del R-Tree.	20
20	MINDIST y MAXDIST entre un punto y un MBR.	20
21	R-Tree con tres hojas y consulta por radio (q, r)	21
22	R-Tree con tres hojas y consulta por rectángulo (L, U)	21
23	R-Tree con tres hojas y consulta por polígono.	22
24	Ray casting dentro del MBR del polígono.	22
25	k-NN con $k = 2$ sobre el árbol de la figura 21.	23
26	Hojas de una carga masiva STR con $n = 27$ y $M = 3$	24
27	External sort con B páginas de buffer.	26
28	Costo del external sort con $B = 64$. Los saltos ocurren en $T = B$, $B(B - 1) = 4032$ y $B(B - 1)^2 = 254016$ páginas. El external hash cuesta 3 desde la primera página, porque no tiene el atajo de memoria del sort.	26
29	External hash: particionamiento con h_p y procesamiento por partición para GROUP BY y hash join.	27
30	AST del SELECT del listado 2, con las clases de <code>query/ast.py</code>	29
31	Orden en que <code>ejecutar_select</code> de <code>query/conexion.py</code> elige la ruta de acceso. GROUP BY y el external sort se aplican después sobre las filas obtenidas.	32
32	Patrones SQL que activan el R-Tree.	35
33	Conversión del plan textual al árbol del PlanPanel: cada paso se vuelve un nodo, el primero queda como hoja y <code>Projection</code> es la raíz.	36
34	Escenario <code>escenario_deadlock</code> de <code>transaction/simulation.py</code> : dos sesiones insertan en <code>estudiantes</code> y <code>cursos</code> en orden cruzado. El grafo wait-for se indexa por sesión.	37
35	Ciclo de vida de una transacción explícita y su lista de undo.	38
36	Salida completa del escenario de deadlock.	40
37	Distribución funcional de la interfaz.	41
38	Paneles de archivos y de consultas.	42
39	Paneles de resultados y de plan de ejecución.	43
40	Flujo de una interacción espacial.	44
41	Datos del cuadro 8.	46
42	Tiempo de reorganización del Sequential File (datos del cuadro 9).	47
43	Tiempo de construcción de los índices.	48
44	Espacio en disco de cada índice.	48
45	Búsqueda por igualdad sobre <code>carrera</code> , a la izquierda, y por clave primaria, a la derecha.	49
46	Búsqueda por rango sobre el 1 % y el 50 % de la tabla.	49
47	ORDER BY devolviendo todas las filas.	49
48	Tiempo por operación y operaciones por segundo con inserciones y eliminaciones intercaladas.	51

49	Tiempo de construcción del R-Tree y de GiST.	53
50	Espacio en disco del R-Tree y de GiST.	53
51	Consultas por radio (1, 5 y 10 km).	53
52	Consultas k-NN ($k = 10, 50, 100$).	53

Listings

1	k-NN best-first.	22
2	Consulta relacional sobre la tabla de demostración estudiantes	29
3	DDL, DML y JOIN con las tablas de demostración.	29
4	Consultas espaciales.	34
5	Consultas equivalentes en PostGIS.	52

1. Introducción

Los gestores de bases de datos ocultan casi todas sus decisiones internas. Este proyecto construye desde cero un minigestor de base de datos, para datos relacionales y espaciales, que aplica los conceptos del curso sobre arquitectura, organización de archivos e indexación en una implementación que se puede observar y medir.

Para los datos relacionales implementa Heap File y Sequential File paginado, índices B+ agrupado, B+ no agrupado y Hash Dinámico (Extendible Hashing), algoritmos de memoria externa para ORDER BY, GROUP BY y JOIN, un parser SQL, transacciones con locks y una interfaz con paneles de archivos, consultas, resultados y plan de ejecución. Para los datos espaciales agrega un R-Tree con consultas por radio, k-NN e intersección con polígonos, con distancia euclidiana y Haversine, su extensión SQL y un mapa con los resultados. Cada técnica se compara experimentalmente con sus alternativas. Los enlaces al repositorio y al despliegue están en la [sección de Anexos](#).

Objetivo general. Diseñar e implementar un sistema de gestión de bases de datos persistente que integre almacenamiento, indexación, procesamiento de consultas, concurrencia y visualización, y evaluar sus decisiones con experimentos reproducibles.

Objetivos específicos.

1. Implementar Heap File con reutilización de espacio libre y Sequential File paginado con inserción ordenada, eliminación lazy y reorganización.
2. Implementar índices B+ agrupado, B+ no agrupado y Hash Dinámico (Extendible Hashing), y combinarlos en un mismo WHERE con bitmaps de RIDs.
3. Implementar ORDER BY con External Sorting (k-way merge), y GROUP BY y JOIN con External Hashing o con índices.
4. Implementar un parser SQL para SELECT, INSERT y DELETE, con su extensión espacial.
5. Soportar BEGIN TRANSACTION, END TRANSACTION y ROLLBACK con locks, y demostrar con una simulación con hilos cómo el sistema maneja las race conditions.
6. Implementar un R-Tree con consultas por rango, k-NN e intersección con polígonos, con distancia euclidiana y Haversine.
7. Construir una interfaz con paneles de archivos, consultas, resultados, plan de ejecución y mapa.
8. Comparar experimentalmente Heap File y Sequential File, los tres índices relacionales, y la búsqueda secuencial, el R-Tree y el GiST de PostgreSQL.

2. Arquitectura del sistema

2.1 Capas y módulos

La [figura 1](#) muestra las capas del sistema. Una consulta baja desde la interfaz hasta la memoria secundaria, y cada capa corresponde a un directorio del repositorio.

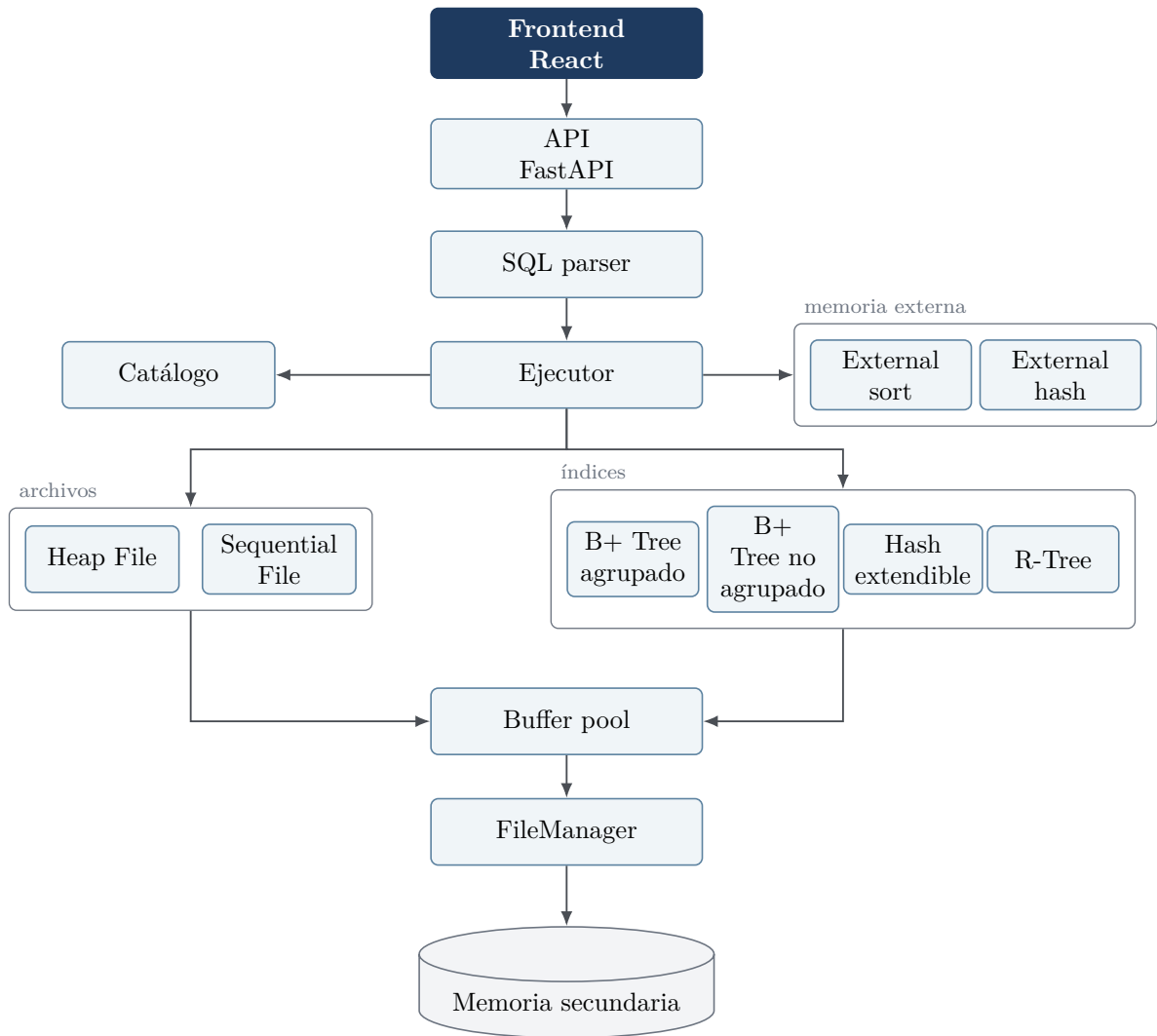


Figura 1. Arquitectura por capas del minigestor. El ejecutor consulta el catálogo, selecciona algoritmos externos, archivos o índices, y delega el acceso físico al buffer pool y al FileManager.

- El frontend (`frontend/`) es una aplicación React con los paneles de archivos, consultas, resultados, plan de ejecución, inspector físico y mapa.
- La API (`backend/`) recibe el SQL por HTTP con FastAPI y lo pasa al motor a través de `EngineAdapter`, que también arma el plan que se muestra en la interfaz.
- El SQL parser (`query/lexer.py`, `query/parser.py`, `query/ast.py` y `query/semantic.py`) convierte el texto en un árbol sintáctico y valida tablas, columnas y tipos.
- El ejecutor (`query/conexion.py`) elige cómo resolver cada sentencia y la ejecuta. Consulta el catálogo (`query/catalog.py`), que registra las tablas, su organización y sus índices, y recurre a los algoritmos de memoria externa (`engine/external.py`) para ORDER BY, GROUP BY y JOIN. Antes de ejecutar, toma los locks de la tabla a través de `transaction/`.
- Los archivos (`storage/`) son el Heap File y el Sequential File, y los índices (`index/`) son el B+ Tree agrupado, el B+ Tree no agrupado, el hash extendible y el R-Tree.

- El buffer pool y el FileManager (**engine/**) leen y escriben las páginas en la memoria secundaria ([apartado 4](#)).

El directorio **common/** reúne lo que comparten todas las capas, que son los tipos de datos, la slotted page, el formato de los registros y las funciones geométricas.

2.2 Flujo de una consulta

1. El frontend envía SQL y un `session_id` a `POST /api/query`.
2. El lexer produce tokens; el parser construye un AST y el analizador semántico valida tablas, columnas, tipos e índices admisibles.
3. El gestor transaccional adquiere un lock S o X a nivel de tabla.
4. El planificador por reglas selecciona índice, búsqueda secuencial, escaneo completo o algoritmo externo.
5. El almacenamiento traduce filas a RIDs, páginas y operaciones físicas.
6. El backend mide tiempo real de pared, convierte el plan textual en un árbol didáctico y devuelve filas, estado transaccional y metadatos.
7. La interfaz presenta la tabla, el plan, el inspector físico o el mapa y conserva la sesión en `localStorage`.

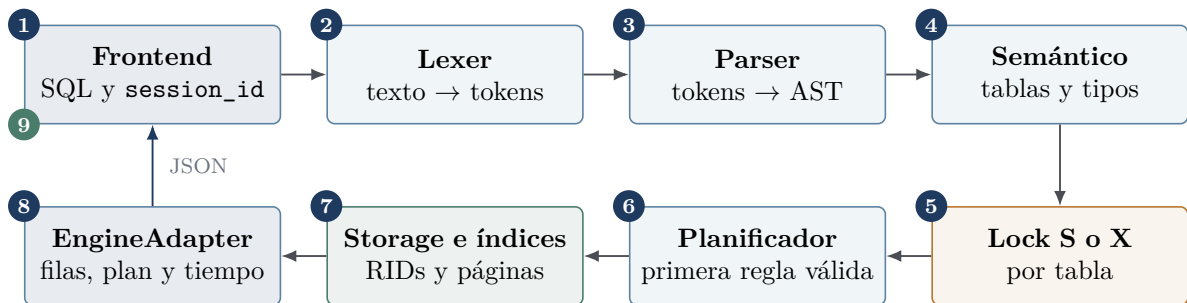


Figura 2. Recorrido de una sentencia SQL por los módulos del sistema.

3. Dominio de datos

El sistema trabaja con un dominio universitario para la parte relacional y con comercios de Lima para la parte espacial. Al arrancar, **EngineAdapter** registra tres tablas de demostración, que el usuario puede consultar sin cargar nada ([cuadro 1](#)).

Las dos primeras se definen en el código del backend, y **tiendas** se crea con sentencias SQL, así que queda registrada en `data/catalog.json` como cualquier tabla del usuario ([apartado 8](#)). Las tablas cubren los tipos que interesa ejercitar: claves enteras, textos de largo variable, un `FLOAT`, un `SMALLINT` y un `POINT`, y una tabla de cada organización de archivo.

Cuadro 1. Tablas de demostración que crea el backend.

Tabla	Archivo	Esquema	Índices
estudiantes	Heap	id INT PK, nombre VARCHAR(40), carrera VARCHAR(30), promedio FLOAT	B+ no agrupado en id, hash en carrera
cursos	Sequential, clave codigo	codigo INT PK, titulo VARCHAR(50), creditos SMALLINT, departamento VARCHAR(20)	B+ agrupado en codigo
tiendas	Heap	id INT PK, nombre VARCHAR(40), categoria VARCHAR(20), ubicacion POINT	B+ único en id, R-Tree en ubicacion

Datos relacionales. `data/generate_data.py` regenera `estudiantes` y `cursos` con 1 000, 10 000 o 100 000 registros y reconstruye sus índices. Los estudiantes tienen IDs consecutivos desde 1, un nombre formado por un nombre y un apellido tomados de listas fijas, una de ocho carreras y un promedio uniforme entre 11 y 19,8. Los cursos tienen códigos consecutivos desde 101, un título que combina una de 25 áreas con un nivel, sus créditos y su departamento. Cada generador usa una semilla fija que depende del tamaño, así que cada dataset es reproducible.

Datos espaciales. `common/datos_lima.py` genera las tiendas, y lo usan tanto el backend como el generador y el benchmark espacial. Define veinte distritos de Lima, cada uno con su centro y un peso relativo. Cada tienda elige un distrito según esos pesos y se ubica alrededor de su centro con una desviación normal de 0,012 grados por eje, unos 1,3 km, recortada al rectángulo de Lima Metropolitana entre las latitudes $-12,30$ y $-11,85$ y las longitudes $-77,20$ y $-76,80$. Su categoría se elige entre diez, como farmacia, restaurante o grifo. El resultado es una nube de puntos concentrada por distrito, más parecida a una ciudad real que una distribución uniforme. El mismo módulo genera los centros de las consultas del benchmark y define el punto de referencia de UTEC en Barranco, $(-12,1354; -77,0224)$, que usan los ejemplos de este informe.

4. Gestión de almacenamiento

Todos los archivos del sistema, de datos y de índices, se leen y escriben en páginas de 4096 bytes a través de tres capas. Cada capa habla solo con la de abajo, así que ninguna estructura abre un archivo por su cuenta.

4.1 FileManager

El FileManager es la única capa que toca los archivos del sistema operativo. Las demás piden la página i de un archivo, y el FileManager mantiene los archivos abiertos, ubica la página en el byte $i \cdot 4096$ y lee o escribe exactamente una página. Cada escritura se entrega al sistema operativo en el momento, sin quedar retenida en el buffer interno de Python.

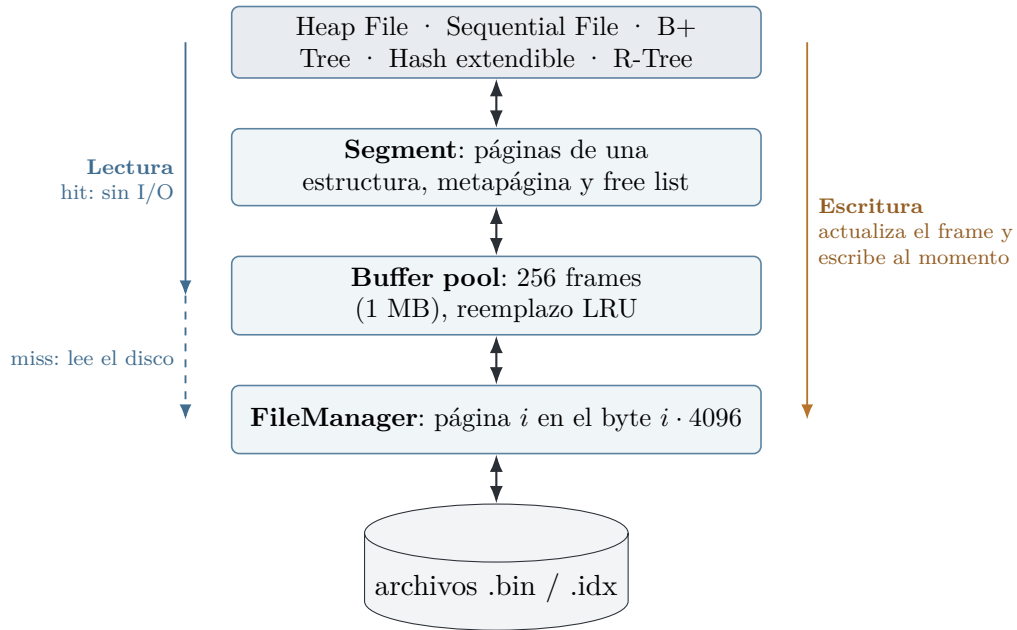


Figura 3. Capas de almacenamiento compartidas por todas las estructuras.

4.2 Buffer pool

El buffer pool guarda en memoria las páginas leídas para no volver a leerlas del disco. Tiene 256 frames, es decir 1 MB, y cuando se llena reemplaza la página usada hace más tiempo (LRU, figura 4). Un solo buffer pool es compartido por todas las estructuras.

En un buffer pool clásico, quien pide una página recibe una referencia al frame donde está y la usa directamente en memoria. Mientras la usa, la página queda fijada (*pinned*), con un contador de cuántos la están usando, y el reemplazo solo puede elegir frames con el contador en cero. Las escrituras también se hacen sobre el frame, que queda marcado como sucio (*dirty*). La página recién se escribe al disco, lo que se llama *flush*, cuando el frame se reemplaza o en un checkpoint, así que varias modificaciones a la misma página cuestan una sola escritura.

Este buffer pool cambia las dos cosas. No usa pins, porque no entrega referencias sino los bytes de la página, que no se pueden modificar y que cada estructura convierte a sus propios objetos. Nadie apunta a un frame, así que cualquiera puede desalojarse en cualquier momento. Tampoco retrasa las escrituras. Cada escritura pasa al FileManager en el momento, que la escribe y hace flush de esa página, y además actualiza el frame, porque las estructuras suelen leer, modificar y volver a leer la misma página. Así, ningún frame tiene una versión más nueva que el archivo, no hace falta la marca de sucio y reemplazar un frame nunca cuesta una escritura.

Se descartó retrasar las escrituras porque las páginas llegarían al archivo en el orden del LRU y no en el orden en que la operación las escribió. Por ejemplo, el Extendible Hash escribe la página de overflow antes que el bucket que apunta a ella, así que si el proceso muere entre las dos escrituras queda en el archivo una página que nadie referencia, pero nunca un bucket apuntando a una página que no existe. Con escrituras retrasadas, el bucket podría llegar al archivo antes que su overflow, y una lectura posterior seguiría el

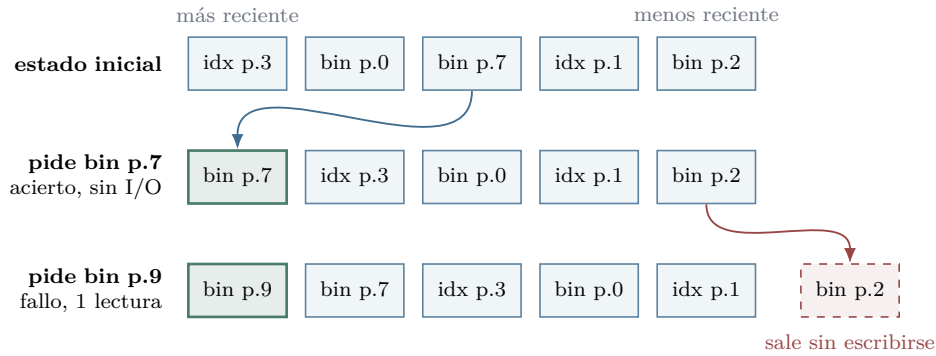


Figura 4. Reemplazo LRU en un buffer pool de cinco frames.

puntero hasta una página vacía o con contenido viejo. Además, el rollback de las transacciones ([apartado 9](#)) lee la versión anterior de cada página desde el archivo justo antes de sobrescribirla, así que necesita que el archivo esté al día.

El trade-off es que cada escritura cuesta un I/O en el momento. Una serie de inserciones sobre la misma página la escribe una vez por inserción, cuando con escrituras retrasadas se escribiría una sola vez al desalojarla. A cambio, desalojar no cuesta I/O y, si el proceso termina, no se pierde ninguna escritura ya hecha.

4.3 Segment

En otros gestores, como Oracle e InnoDB, un segment es el espacio de almacenamiento que pertenece a un objeto, como una tabla o un índice. Aquí un Segment es la vista de un archivo como una secuencia de páginas de una sola estructura. Lee, escribe y agrega páginas a través del buffer pool, y además asigna y libera páginas. En una primera versión del proyecto, sin Segment, cada índice tenía sus propias funciones para leer y escribir su archivo y su propia free list, con el mismo código repetido en cada estructura. Al implementar el Segment, la gestión de páginas libres quedó escrita una sola vez y ninguna estructura accede a su archivo por fuera del buffer pool, lo que evita que el pool conserve frames con una versión vieja de una página. Todas las estructuras de archivo usan el Segment, pero solo los índices liberan páginas. El Heap File y el Sequential File reutilizan el espacio de los registros eliminados con sus propias estrategias, que se describen en [apartado 5](#). En un índice, en cambio, eliminar claves puede dejar una página completa sin uso, cuando dos nodos se combinan en uno o cuando una página de overflow del hash queda vacía.

Las páginas liberadas forman una free list encadenada dentro de ellas mismas ([figura 5](#)). El Segment conoce la cabeza de la lista, cada página libre guarda en sus primeros 4 bytes el número de la siguiente y la última guarda -1, así que la lista no ocupa espacio extra. Al pedir una página nueva se toma la cabeza, y solo cuando la lista está vacía crece el archivo.

El Segment mantiene la cabeza en memoria y no decide dónde se guarda en el archivo. Cada índice la escribe en su metapágina, que es la página 0 de su archivo y reúne lo que la estructura necesita para abrirse, como la raíz del B+ Tree o la global depth del hash.

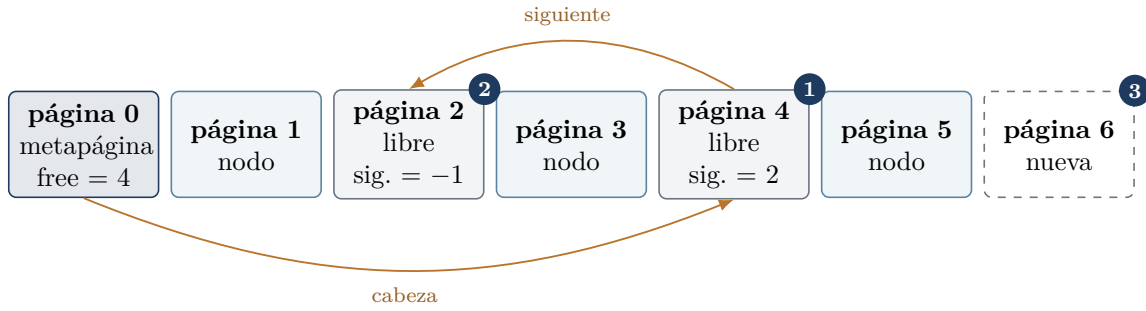


Figura 5. Free list de un archivo de índice y orden en que se asignan las páginas.

4.4 Slotted page, registro y RID

Los archivos de datos usan slotted pages de 4096 bytes. La página se divide en cuatro zonas. Al inicio está la cabecera, de 4 bytes. Le sigue el directorio de slots, que tiene un slot por registro y crece hacia el final de la página a medida que se agregan registros. Los registros se escriben desde el final de la página hacia el inicio, así que el espacio libre queda en medio, entre el directorio y los registros (figura 6). La cabecera guarda dos valores, la cantidad de slots n y el byte f donde empieza el primer registro, que es también donde termina el espacio libre.

Cada slot ocupa 4 bytes y guarda el offset o_i y el largo ℓ_i del registro i . Como la cabecera y los slots tienen tamaño fijo, la posición del slot i y el espacio libre se calculan directamente:

$$\text{slot}_i = 4 + 4i, \quad \text{registro}_i = [o_i, o_i + \ell_i), \quad \text{libre} = f - (4 + 4n).$$

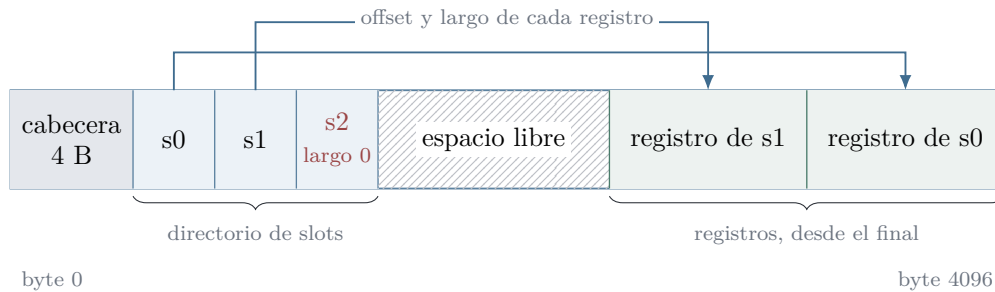


Figura 6. Slotted page de un archivo de datos.

Un registro se identifica por su RID (p, i) , donde p es la página e i el número de slot. El RID no guarda la posición del registro. Para leerlo se lee la página p , se toma el slot en el byte $4 + 4i$ y de ahí se obtienen o_i y ℓ_i . Al borrar un registro su slot queda con $\ell_i = 0$ y puede reutilizarse en una inserción posterior, pero no se elimina del directorio, así que los demás slots conservan su número.

Con el tiempo los borrados dejan huecos entre los registros, y cuando una inserción no cabe en el espacio libre la página se compacta (figura 7). Los registros se mueven para quedar juntos y cada slot actualiza su o_i , pero ningún slot cambia de número, así que los RID guardados en los índices siguen siendo válidos.

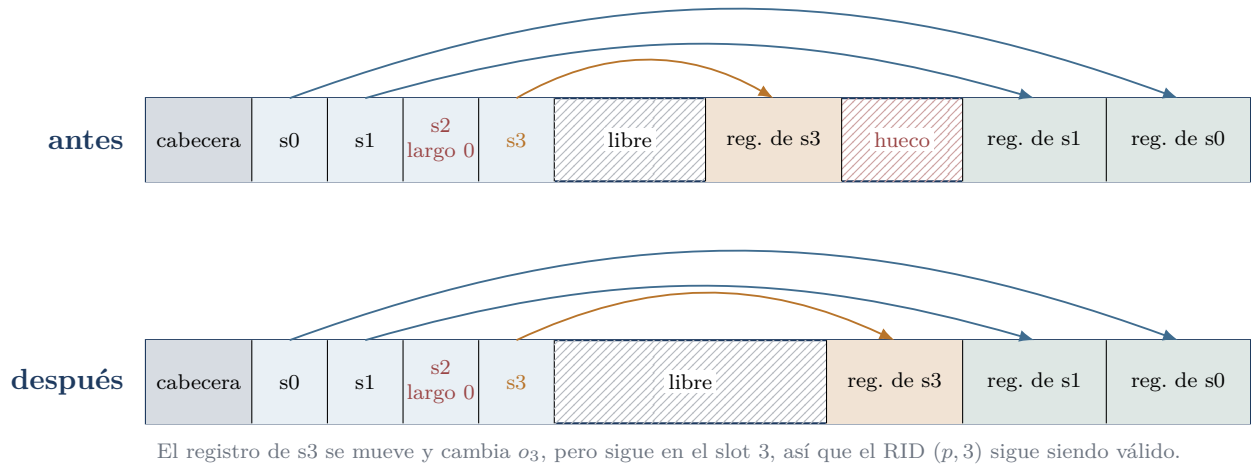


Figura 7. Compactación de una slotted page. El hueco que dejó el registro borrado de s2 se junta con el espacio libre.

5. Organización de archivos

5.1 Heap File

El Heap File es una secuencia de slotted pages sin metapágina, y cada registro se identifica por su RID (página, slot). Para reutilizar espacio, el archivo mantiene en memoria un mapa de espacio libre formado por el conjunto de páginas reutilizables (con bytes libres o con algún slot borrado), los bytes libres de cada página y el conjunto de páginas con slots borrados.

Para simplificar, los costos de este informe cuentan solo I/O de página, donde cada lectura o escritura de una página cuenta como un I/O. No se considera el trabajo en memoria ni el buffer pool. B es el número de páginas del archivo.

Búsqueda por igualdad. Como el archivo no mantiene orden, buscar por clave exige recorrer el archivo. Si la clave existe se lee en promedio la mitad, y si no existe se lee completo, así que el costo es $\mathcal{O}(B)$.

Búsqueda por rango. Los registros que cumplen el rango pueden estar en cualquier página, así que el costo es B .

Inserción. Se prueba primero la última página. Si el registro no cabe, se recorren las páginas reutilizables que tienen espacio suficiente o algún slot borrado, ordenadas de mayor a menor espacio libre, y si ninguna sirve se agrega una página. Dentro de la página, un slot borrado se reutiliza y la página se compacta antes de declarar que no hay espacio. Tras cada intento se actualiza el mapa. Si el registro cabe en la última página, el insert cuesta 2 I/O. Si hay que probar c páginas candidatas, cuesta $\mathcal{O}(c)$.

Eliminación. El registro se busca por clave y se vacía su slot, y la página entra al mapa como reutilizable. Cuesta la búsqueda más la escritura de la página, $\mathcal{O}(B)$.

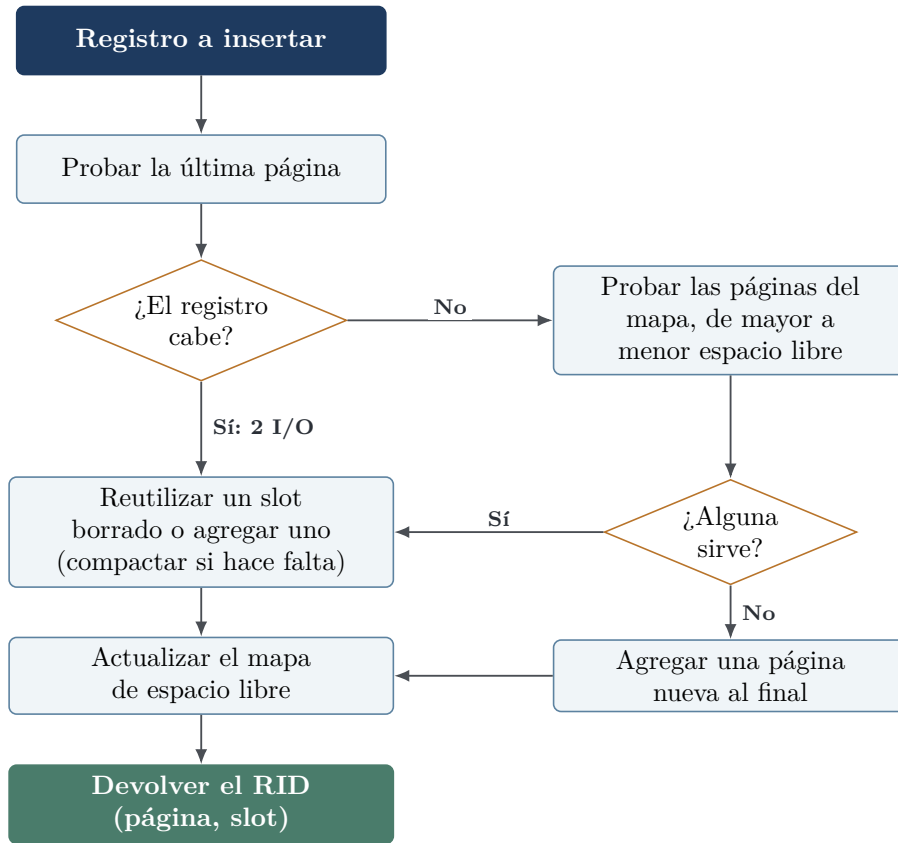


Figura 8. Inserción en Heap File con reutilización de espacio.

Reapertura. El mapa no se persiste. El primer insert que lo necesita después de abrir el archivo lo reconstruye recorriendo todas las páginas, con un costo único de B .

5.2 Sequential File

El Sequential File mantiene los registros ordenados por la clave primaria. Se compone de dos archivos de slotted pages, MAIN y AUX. MAIN queda ordenado después de cada reorganización, y AUX recibe en orden de llegada las claves que caen entre registros de MAIN. Una lista enlazada recorre ambos archivos en orden de clave, de modo que leer la tabla ordenada no exige reescribir el archivo en cada inserción. Un ORDER BY por la clave, por ejemplo, se resuelve siguiendo esa lista. Toda tabla secuencial tiene además un B+ agrupado sobre su clave primaria ([apartado 6](#)), que se crea junto con la tabla.

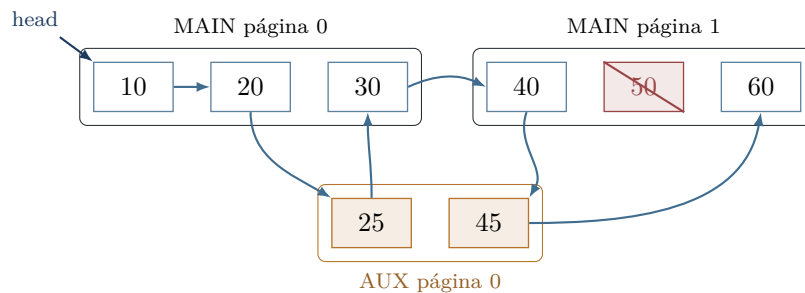


Figura 9. Lista enlazada del Sequential File en orden de clave, a través de MAIN y AUX.

En esta sección B es el número de páginas de MAIN y A el de AUX.

Entry. Cada slot guarda una cabecera de 10 bytes (`<bii?>`) con el archivo, la página y el slot del sucesor y la marca de borrado, seguida del registro.

RID. Los RID de AUX se codifican con página negativa,

$$RID_{MAIN}(p, s) = (p, s), \quad RID_{AUX}(p, s) = (-(p + 1), s),$$

así que un RID indica por sí solo en qué archivo está el registro. Los índices no agrupados guardan RIDs de cualquiera de los dos archivos sin un campo extra, y el B+ agrupado, que es disperso sobre las páginas de MAIN, usa el signo para no registrar los registros de AUX.

Metadatos. Un archivo de 30 bytes guarda la cabeza y la cola de la lista y los contadores de registros en AUX, vivos y borrados. Para no escribirlo en cada operación se mantiene en memoria y se escribe cada 256 modificaciones, y mientras haya cambios sin guardar existe un archivo marcador `.dirty`. Si el marcador existe al abrir, los metadatos en disco están desactualizados y se reconstruyen recorriendo el archivo.

Búsqueda por igualdad. Se usa el B+ agrupado para llegar a la página de MAIN que puede contener la clave, y luego se recorre AUX. Como la reorganización mantiene $A \leq \lfloor \log_2 B \rfloor$, el costo es

$$\mathcal{O}(\log_m B) + \mathcal{O}(\log_2 B) = \mathcal{O}(\log B),$$

con m el orden del B+.

Búsqueda por rango. El B+ agrupado ubica la primera página de MAIN del rango, y desde ahí se leen páginas consecutivas de MAIN hasta pasar el límite superior. AUX se recorre completo y sus resultados se mezclan en orden con los de MAIN. Si K es el número de páginas de MAIN que contienen resultados, el costo es $\mathcal{O}(\log B) + K$.

Inserción. Hay dos casos según la clave nueva k . Si k es mayor que la última clave del archivo, el registro se agrega al final de MAIN y el registro que era el último de la lista pasa a apuntar a él. En otro caso, se desciende por el B+ agrupado hasta la página de MAIN que contiene al predecesor de k , es decir, al último registro con clave menor que k . A partir de ese registro se recorre la lista hasta la primera clave mayor que k . Si en ese tramo aparece k , el registro se rechaza, porque la clave del archivo es la clave primaria. Si no aparece, el registro se agrega a AUX y se reencadena entre su predecesor y su sucesor. En los dos casos se escribe la entrada nueva y se reescribe la de su predecesor, 2 I/O, y con el descenso por el árbol el insert cuesta $\mathcal{O}(\log_m B)$.

Eliminación. La eliminación es lazy. Como en el insert, se desciende por el B+ agrupado hasta la página del predecesor y a partir de él se recorre la lista hasta la clave. El registro se marca como borrado y se desenlaza de la lista, reescribiendo su entrada y la de su predecesor, así que cuesta lo mismo que el insert, $\mathcal{O}(\log_m B)$. Quitarlo físicamente no

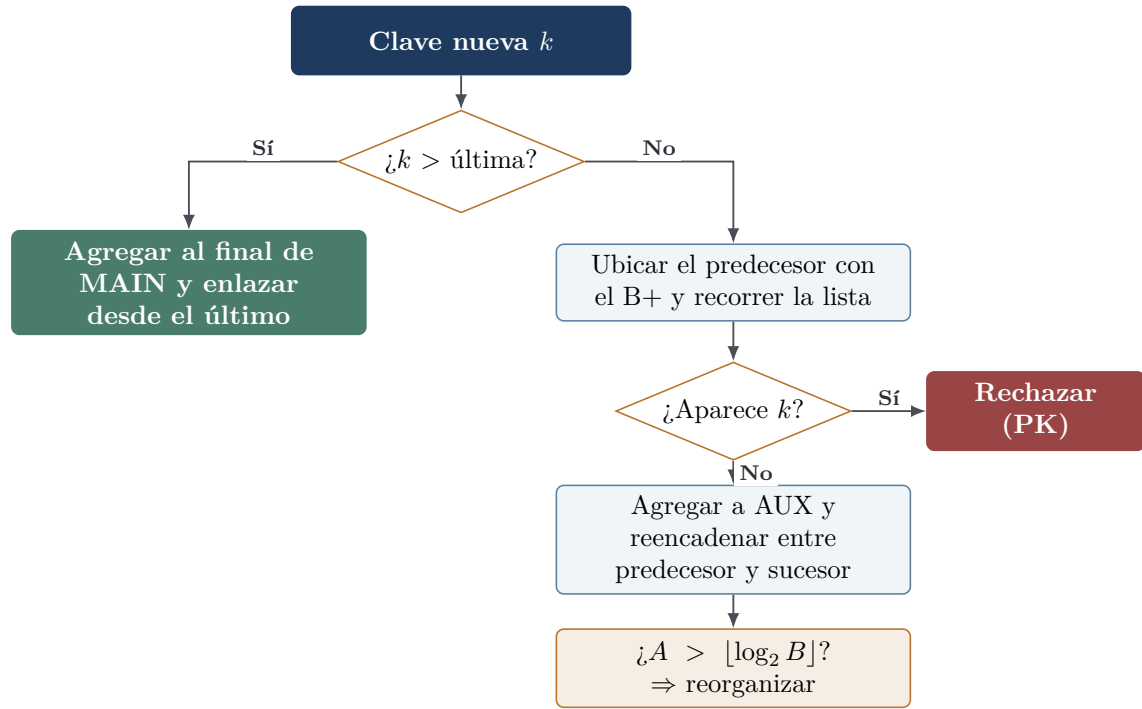


Figura 10. Inserción en el Sequential File y verificación de reorganización.

serviría de nada, porque MAIN solo recibe al final claves mayores que la última y toda clave intermedia va a AUX, de modo que el hueco quedaría vacío hasta la siguiente reorganización, aunque se volviera a insertar la misma clave. Además, la entrada marcada conserva su clave, y así la página mantiene el rango de claves que conoce el B+ agrupado. El espacio de los registros borrados se recupera todo junto al reorganizar.

Reorganización. Después de cada inserción, borrado o actualización se revisa si AUX tiene más de $\lfloor \log_2 B \rfloor$ páginas o si más del 30 % de las entradas de MAIN y AUX están marcadas como borradas, y en ese caso el archivo se reorganiza. La primera condición solo puede activarse al insertar y la segunda solo al borrar. La reorganización sigue la lista enlazada para obtener los registros vivos en orden de clave y con ellos arma, página por página, un nuevo MAIN en un archivo temporal, donde cada registro apunta al siguiente (figura 11). Al terminar, el archivo temporal reemplaza a MAIN y AUX se vacía.

Sea n el número de registros vivos. El nuevo MAIN guarda solo los registros vivos, así que ocupa a lo sumo las $B + A$ páginas de los archivos anteriores, y escribirlo cuesta a lo sumo $B + A$ I/O. Para leer, la lista salta entre MAIN y AUX, así que con el modelo de costos de este informe, que no considera el buffer pool, cada registro cuesta la lectura de su página, y la reorganización cuesta $\mathcal{O}(n)$.

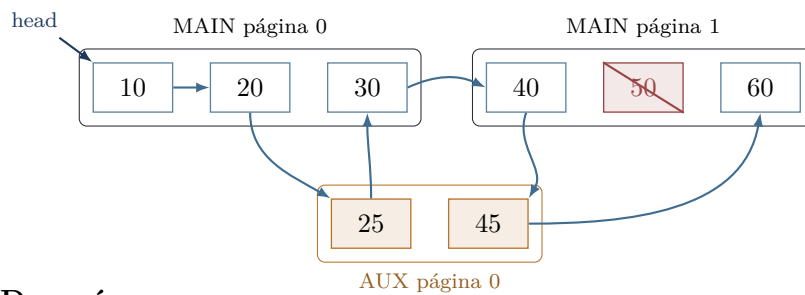
Con el buffer pool el costo real es menor, y la diferencia está en cómo la lista recorre cada archivo. MAIN se recorre en orden. La lista puede salir de una página de MAIN hacia AUX, pero vuelve a la misma página, que se acaba de usar y sigue en el pool, y cuando la deja no vuelve a ella. Así cada página de MAIN se lee del disco una sola vez. AUX, en cambio, se visita salteado durante todo el recorrido, porque sus registros llegaron en cualquier orden y cada página tiene claves de todo el rango. Si las páginas de AUX

permanecen en el pool hasta el final, también se leen una sola vez, y la lectura cuesta $B + A$ I/O, igual que la escritura. Como $A \leq \lfloor \log_2 B \rfloor$, la reorganización cuesta $\mathcal{O}(B)$. Si el pool no alcanza para retenerlas, cada registro de AUX puede costar una lectura. AUX tiene a lo sumo $\lfloor \log_2 B \rfloor$ páginas, cada una con unos n/B registros, así que la reorganización cuesta

$$\mathcal{O}\left(B + \frac{n}{B} \log B\right).$$

El cociente n/B es la cantidad de registros que caben en una página, y no depende del tamaño de la tabla, sino del tamaño de la página y del registro. Si la tabla crece, n y B crecen en la misma proporción y n/B se mantiene constante, así que el costo se reduce a $\mathcal{O}(B + \log B) = \mathcal{O}(B)$.

Antes



Después

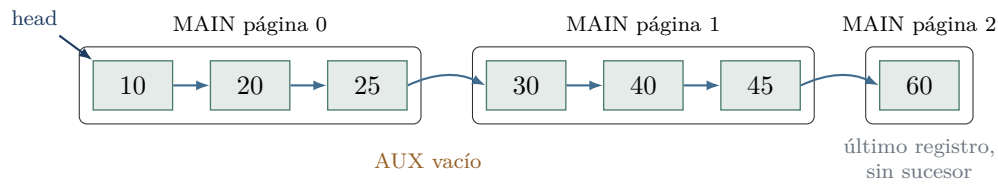


Figura 11. Reorganización del Sequential File con 3 registros por página.

5.3 Comparación

El [cuadro 2](#) resume los costos de ambas organizaciones. El Heap File inserta en tiempo constante, pero buscar o eliminar una clave le exige recorrer el archivo. El Sequential File paga $\mathcal{O}(\log B)$ en cada operación a cambio de mantener el orden, que además le permite resolver búsquedas por rango y lecturas ordenadas sin recorrer todo el archivo.

Operación	Heap File	Sequential File
Búsqueda por igualdad	$\mathcal{O}(B)$	$\mathcal{O}(\log B)$
Búsqueda por rango	B	$\mathcal{O}(\log B) + K$
Insertión	2 o $\mathcal{O}(c)$	$\mathcal{O}(\log B)$
Eliminación	$\mathcal{O}(B)$	$\mathcal{O}(\log B)$

Cuadro 2. Costos en I/O de página.

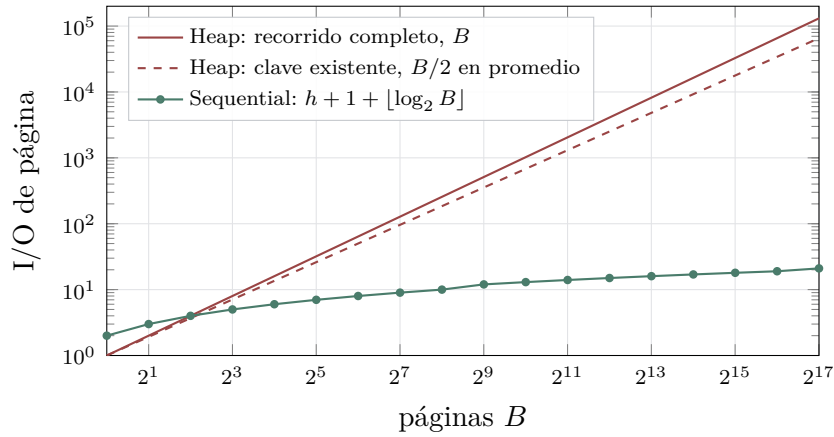


Figura 12. Búsqueda por igualdad en I/O de página, con AUX en su tamaño máximo.

6. Estructuras de indexación

6.1 Codificación de claves y páginas de nodo

Antes de entrar a un índice, cada clave se convierte a bytes. Así el índice almacena y compara bytes sin importar si la columna es INT, FLOAT, CHAR o de otro tipo.

Claves del B+ Tree. El B+ Tree necesita comparar claves para decidir el descenso y resolver rangos, así que `index/key_codec.py` convierte cada valor a bytes que conservan el orden. Comparar dos claves codificadas da el mismo resultado que comparar los valores originales. El tipo de la columna se guarda una sola vez en la metapágina del árbol, porque todas sus claves pertenecen a la misma columna. Cada tipo se codifica así:

- SMALLINT, INT y BIGINT ocupan 2, 4 y 8 bytes en el registro, pero en el índice los tres se amplían a 8 bytes. Ampliarlos no pierde información y deja un solo formato para los enteros, a cambio de algunos bytes extra por clave.
- FLOAT y DOUBLE ocupan 4 y 8 bytes en el registro, y en el índice ambos se guardan como un DOUBLE de 8 bytes, lo que tampoco pierde información. Los bytes del formato IEEE 754 se transforman para que su orden coincida con el orden numérico. El valor -0.0 se normaliza a 0.0 y NaN se rechaza, porque no tiene orden.
- CHAR y VARCHAR se guardan en UTF-8 sin relleno, lo que conserva el orden de los caracteres y deja la cadena `ab` antes que `abc`.
- BOOL ocupa 1 byte.

Una clave codificada puede medir hasta 2030 bytes. Un nodo necesita al menos dos entradas para que un split lo divida en dos nodos no vacíos. De los 4096 bytes de la página, 16 son de cabecera, y cada entrada ocupa 2 bytes de largo, hasta 8 de payload y la clave, así que dos entradas con una clave de K bytes entran si

$$2(2 + 8 + K) \leq 4096 - 16 \implies K \leq 2030.$$

Un `VARCHAR(n)` guarda a lo sumo *n* bytes, así que una columna con $n \leq 2030$ siempre se puede indexar, y en una más ancha se rechaza la inserción de un valor que pase el límite.

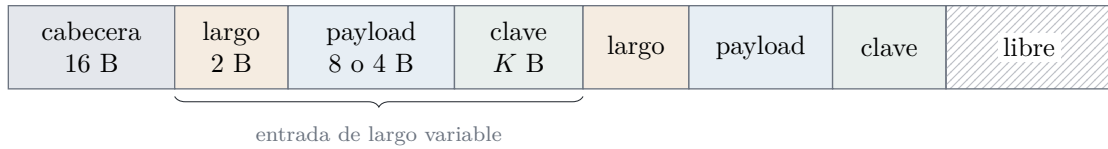
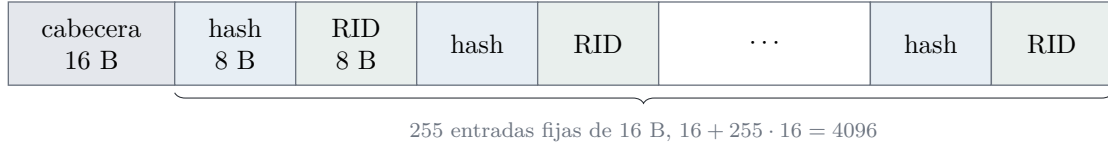
Claves del hash. El hash extendible no necesita orden sino igualdad, así que `index/hash_utils.py` reduce cada clave a un hash de 8 bytes calculado con BLAKE2b. El hash tiene que ser determinista, es decir, dar el mismo resultado para la misma clave en cualquier ejecución, porque el índice persiste en disco. La función `hash` de Python no lo es para strings, porque usa una semilla aleatoria distinta en cada proceso, y un índice construido con ella dejaría de encontrar sus claves al reiniciar. Entre las funciones deterministas se eligió BLAKE2b porque viene en la biblioteca estándar de Python, permite pedir una salida de exactamente 8 bytes y distribuye bien todos sus bits, lo que importa porque el hash extendible usa los bits bajos para elegir el bucket. FLOAT y DOUBLE no se pueden indexar con hash, porque la igualdad de flotantes no es exacta bit a bit (0.0 y -0.0 son iguales y sus bytes no, y NaN no es igual a sí mismo).

Páginas de índice. Los nodos de los índices no usan slotted page. En un archivo de datos, los índices apuntan a cada registro por su RID, así que el RID debe seguir siendo válido aunque el registro se mueva dentro de la página, y eso es lo que resuelve el directorio de slots. En un índice, en cambio, nada externo apunta a una entrada individual y la página siempre se lee y se escribe completa, así que no hace falta un directorio de slots, ni siquiera con entradas de largo variable, porque basta con guardar el largo de cada una. Además, las entradas deben estar en orden de clave, mientras que la slotted page reutiliza slots en orden de llegada. Por eso cada índice tiene su propio formato de página de 4 KB, con una cabecera y las entradas empaquetadas una tras otra:

- El nodo del B+ Tree (`index/node_page.py`) tiene una cabecera de 16 bytes con `is_leaf`, un byte que indica si el nodo es hoja, la cantidad de entradas, el último hijo y las hojas anterior y siguiente. Cada entrada guarda el largo de la clave (2 bytes), el payload y la clave. En una hoja el payload es un RID (8 bytes) en el índice no agrupado y un `page_id` (4 bytes) en el agrupado, y en un nodo interno es el hijo izquierdo de la clave (4 bytes). Como las claves VARCHAR se guardan sin relleno, las entradas son de largo variable.
- El bucket del hash (`index/bucket_page.py`) tiene una cabecera de 16 bytes con la local depth, el tipo de página (primaria u overflow), la cantidad de entradas y la siguiente página de overflow. Cada entrada mide 16 bytes, el hash (8 bytes) y el RID (8 bytes), así que un bucket guarda exactamente 255 entradas. Las entradas son de tamaño fijo porque se guarda el hash y no la clave.

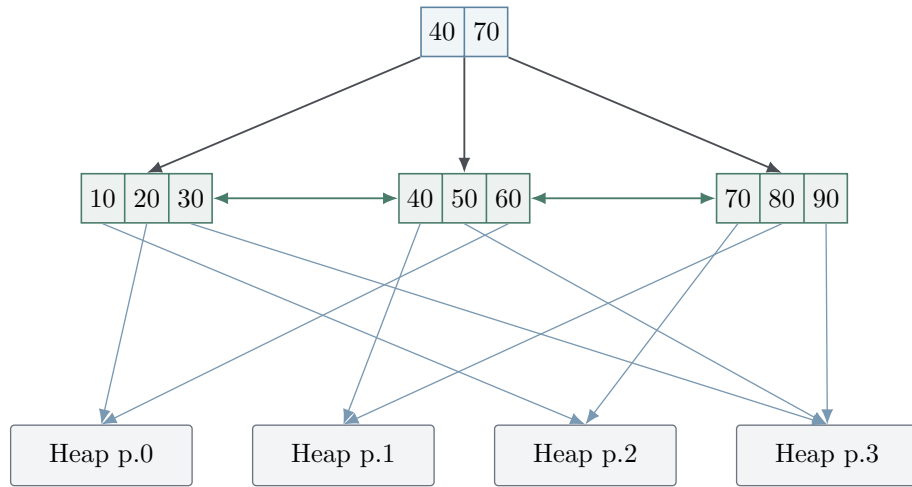
6.2 B+ Tree no agrupado

El B+ Tree no agrupado se crea sobre cualquier columna de una tabla Heap o Sequential con `CREATE INDEX ... USING BTREE`, y toda tabla Heap recibe uno sobre su clave primaria. Sus hojas guardan un par (*clave*, *RID*) por cada fila de la tabla. Un índice puede ser único, en cuyo caso rechaza claves repetidas, como el de la clave primaria, o

Nodo del B+ Tree**Bucket del hash extendible****Figura 13.** Páginas de 4 KB del B+ Tree y del hash extendible.

admitirlas. La metapágina guarda la raíz, la altura, el tipo de clave, si el índice es único y si es agrupado, y la cabeza de la free list.

Cada entrada guarda el largo de la clave, un puntero y la clave. En una hoja el puntero es el RID, y en un nodo interno es el hijo izquierdo, mientras que el último hijo va en la cabecera. Con claves INT, una hoja guarda 226 entradas y un nodo interno tiene 292 hijos. Sea m el número de hijos de un nodo interno. Como el índice tiene una entrada por fila, su altura h es $\mathcal{O}(\log_m n)$, con n el número de filas.

**Figura 14.** B+ Tree no agrupado sobre una tabla Heap.

Búsqueda por igualdad. Se baja desde la raíz eligiendo en cada nodo el primer separador mayor o igual que la clave. Con claves repetidas, un grupo de copias puede quedar repartido entre dos hojas con la misma clave como separador, y bajar con mayor o igual garantiza llegar a la primera copia. En la hoja se recogen las entradas con esa clave, siguiendo la cadena de hojas si continúan en la siguiente. Si k es el número de filas con esa clave, leerlas cuesta a lo sumo B I/O (apartado 8.4.1). El costo es $\mathcal{O}(h + B)$, que en un índice único, donde $k \leq 1$, se reduce a $h + 1$.

Búsqueda por rango. Se baja con el límite inferior y se recorre la cadena de hojas hasta pasar el límite superior. Las hojas recorridas son muchas menos que las k filas del

rango, porque cada una guarda cientos de entradas, y leer las filas cuesta a lo sumo B I/O (apartado 8.4.1). El costo es $\mathcal{O}(h + B)$.

Inserción. Se baja a la hoja que corresponde y se inserta la entrada en su posición. Si entra, se escribe la hoja y ningún nodo interno cambia. Si no entra, la hoja se divide en dos mitades con un número parecido de bytes. La página vieja queda como mitad izquierda, se aloca una página para la mitad derecha, se enlaza en la cadena de hojas, y la primera clave de la mitad derecha sube como separador al padre. Si el padre tampoco tiene espacio se divide igual, y si se divide la raíz el árbol crece un nivel. En un índice único se rechaza una clave que ya existe. El costo es h lecturas y una escritura, y en el peor caso un split por nivel, $\mathcal{O}(h)$.

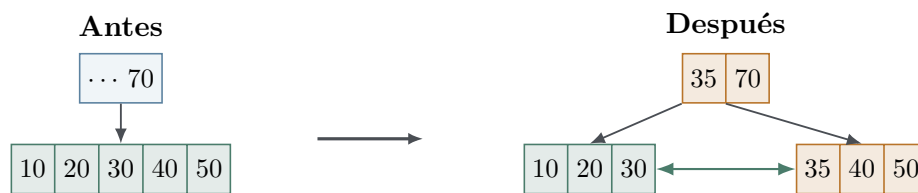


Figura 15. Split de una hoja llena del B+ Tree al insertar 35.

Eliminación. Se baja hasta la hoja y se borra el par $(clave, RID)$. Si la hoja queda con menos de media página ocupada, primero se intenta que un hermano le preste una entrada, actualizando el separador del padre. Si ningún hermano puede prestar, la hoja se fusiona con uno de ellos, se borra el separador del padre y la página liberada vuelve a la free list. La fusión puede repetirse hacia la raíz, así que el costo es $\mathcal{O}(h)$.

Bulk load. Construir el índice sobre una tabla existente, con `CREATE INDEX` o después de reorganizar un Sequential File, no inserta entrada por entrada. Recibe los pares ordenados por clave, llena hojas consecutivas y arma los niveles internos de abajo hacia arriba, escribiendo cada página una vez. Si el índice resultante tiene B páginas, el costo es B .

6.3 B+ Tree agrupado

El B+ Tree agrupado se crea junto con cada tabla Sequential, sobre su clave primaria. Usa las mismas páginas de nodo y el mismo código que el no agrupado, pero guarda una entrada por página de MAIN, con la clave mínima de la página y su `page_id`. Es un índice disperso, así que su tamaño depende del número de páginas de MAIN y no del de registros. Los registros de AUX no se indexan, sino que se recorre AUX, que tiene a lo sumo $\lceil \log_2 B \rceil$ páginas. En esta sección, como en el Sequential File, B es el número de páginas de MAIN y A el de AUX. Como el árbol tiene una entrada por página de MAIN, su altura h es $\mathcal{O}(\log_m B)$, con m el número de hijos de un nodo interno.

Mantenimiento. El árbol solo crece por el final. Una inserción que cae en AUX no lo cambia, y una que se agrega al final de MAIN y ocupa una página nueva le agrega una entrada. La eliminación marca el registro dentro de su página sin mover ningún otro, así que tampoco cambia el árbol. Si se borra el registro mínimo de una página, su entrada

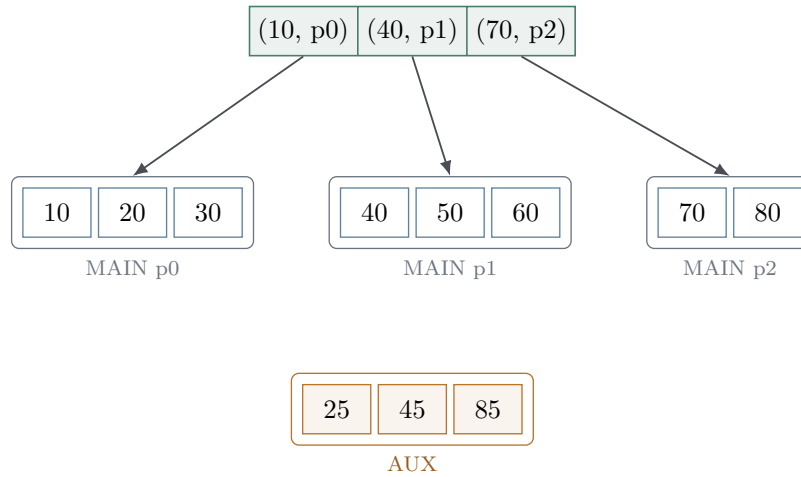


Figura 16. B+ agrupado disperso, con una entrada por página de MAIN.

queda menor que el mínimo real, pero el descenso sigue llevando a la página correcta. Tras cada reorganización el árbol se reconstruye con bulk load sobre el nuevo MAIN.

Búsqueda por igualdad. Se busca la última entrada con clave menor o igual que la buscada, se lee esa página de MAIN y, si la clave no está ahí, se recorre AUX. El costo es $h + 1 + A$, y como $A \leq \lfloor \log_2 B \rfloor$, es $\mathcal{O}(\log B)$.

Búsqueda por rango. Se ubica la página de MAIN del límite inferior y se leen páginas consecutivas hasta pasar el límite superior, porque el orden del índice es el orden físico del archivo. Los registros de AUX que caen en el rango se mezclan en orden con los de MAIN. Si K es el número de páginas de MAIN con resultados, el costo es $h + K + A$, es decir, $\mathcal{O}(\log B + K)$.

6.4 Hash extendible

El hash extendible es un índice no agrupado para búsquedas por igualdad, y se crea con `CREATE INDEX ... USING HASH` sobre cualquier columna que no sea FLOAT ni DOUBLE. Su archivo tiene tres tipos de página, que son la metapágina (global depth, capacidad del bucket, primera página del directorio y cabeza de la free list), las páginas del directorio y los buckets. El directorio tiene 2^D entradas, con D la global depth, y cada una apunta a un bucket. Se guarda en páginas encadenadas de 1023 entradas, se carga en memoria al abrir el índice y solo se escribe cuando cambia, así que consultarlo no cuesta I/O. Un bucket lleno puede tener una cadena de páginas de overflow, y C es el número de páginas de esa cadena.

Búsqueda por igualdad. Los D bits bajos del hash eligen la entrada del directorio, se leen las C páginas del bucket y se devuelven los RIDs cuyo hash coincide. Si k es el número de registros con esa clave, leerlos cuesta a lo sumo B I/O (apartado 8.4.1). El costo es $\mathcal{O}(C + B)$, que sin overflow y con una clave única se reduce a 2 I/O.

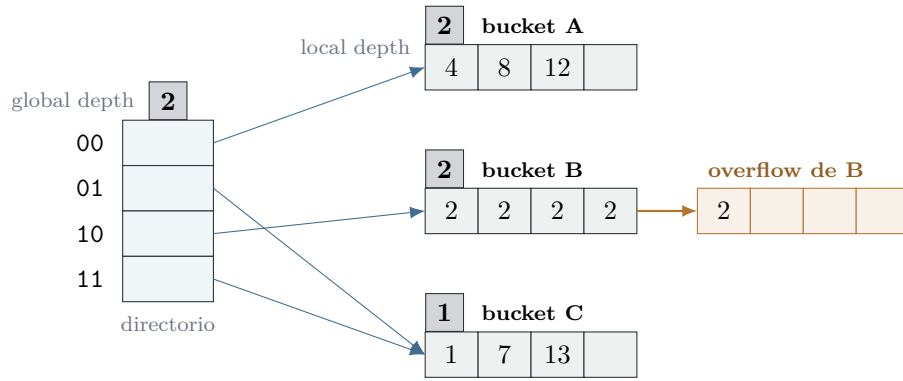


Figura 17. Hash extensible con global depth 2 y capacidad 4 por bucket.

Búsqueda por rango. El hash no conserva el orden, así que no sirve para rangos y el planificador no lo usa en ese caso.

Inserción. Se lee el bucket que indica el directorio, y hay tres casos (figura 18). Si tiene espacio, la entrada se agrega y el bucket se reescribe, con 2 I/O. Si está lleno pero dividirlo no serviría, porque todas sus entradas caerían en la misma mitad, como pasa con muchas claves repetidas, la entrada va a la cadena de overflow. En otro caso el bucket se divide. Si su local depth es igual a D , primero se duplica el directorio, lo que lo reescribe completo. Luego las entradas de las C páginas se reparten entre la página vieja y una nueva según el siguiente bit del hash, se actualiza el directorio y se reintenta la inserción, porque la mitad que recibe la clave puede seguir llena. El costo es $\mathcal{O}(C)$, más la reescritura del directorio cuando se duplica.

Eliminación. Se busca el par $(hash, RID)$ en el bucket y su cadena, se quita y se reescribe la página. Si después esa página y la siguiente de la cadena caben en una sola, se juntan y la otra vuelve a la free list. No hay merge de buckets ni reducción del directorio, porque no mejoran el costo de una búsqueda y con inserciones y eliminaciones alternadas pueden oscilar entre juntar y volver a dividir. El costo es el de la búsqueda más 1 escritura, $\mathcal{O}(C)$.

Bulk load. Para construir el índice sobre una tabla existente se insertan los pares en un archivo nuevo, que reemplaza al anterior al terminar.

6.5 R-Tree

El R-Tree es un índice no agrupado sobre columnas POINT, y se crea con `CREATE INDEX ... USING RTREE`. Sus hojas guardan pares $(punto, RID)$, y cada nodo interno guarda, por cada hijo, el rectángulo mínimo que contiene todos los puntos de ese subárbol. Resuelve consultas por radio y por rectángulo, intersección con polígonos y k-NN.

Modelo espacial y MBR. La tabla guarda un POINT como (lat, lon) , y dentro del árbol se guarda como (x, y) , con $x = lon$ el eje horizontal e $y = lat$ el vertical. El único tipo geométrico del árbol es el MBR (*minimum bounding rectangle*) $R = \{L, U\}$, con $L = (x_{min}, y_{min})$ y $U = (x_{max}, y_{max})$. Un punto es un MBR degenerado, con $L = U$ y área

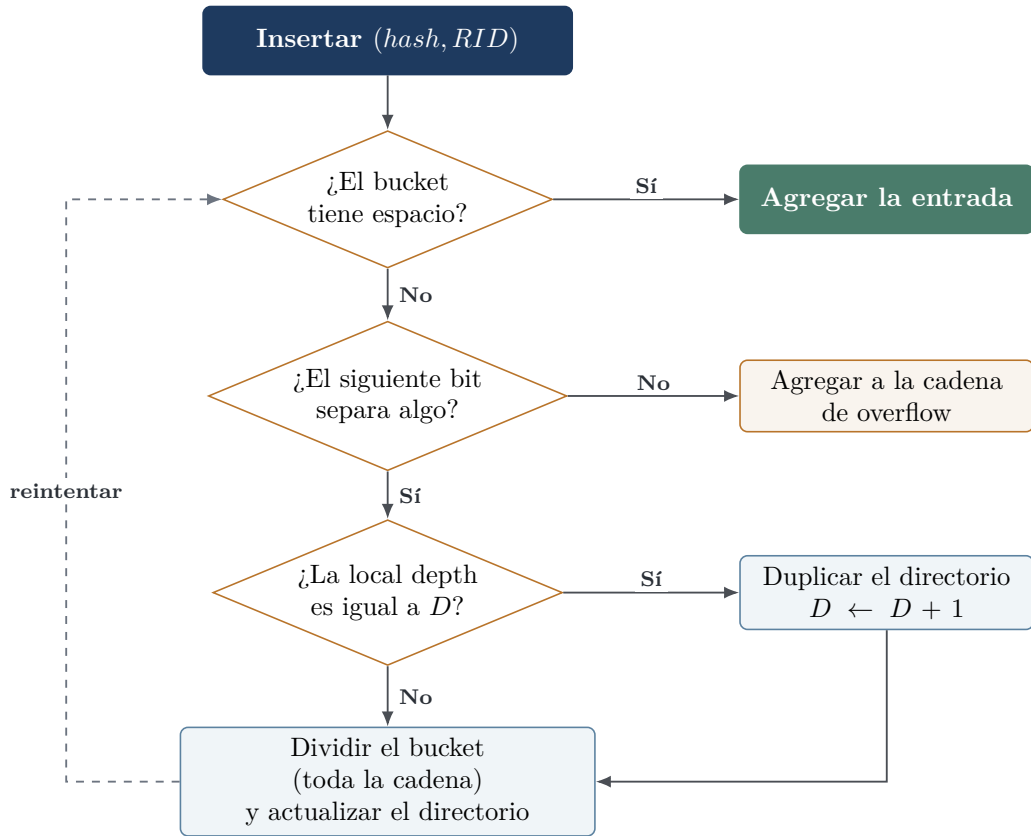


Figura 18. Inserción en el hash extendible.

0, así que la inserción, el split y las distancias tratan igual a los puntos de las hojas y a los rectángulos de los nodos internos.

Formato en disco. Cada nodo es una página de 4 KB con una cabecera de 4 bytes, que indica si es hoja y cuántas entradas tiene, y un arreglo de entradas de tamaño fijo (figura 19). Una entrada de hoja guarda el punto (dos DOUBLE de 8 bytes) y el RID (8 bytes) en 24 bytes, y una entrada interna guarda el MBR (cuatro DOUBLE) y el hijo (4 bytes) en 36 bytes. Sea M el número máximo de entradas de un nodo, que es $\lfloor 4092/24 \rfloor = 170$ en una hoja y $\lfloor 4092/36 \rfloor = 113$ en un nodo interno. La hoja guarda solo el punto y no un MBR degenerado, porque el MBR ocuparía 32 bytes en vez de 16, la entrada pasaría a 40 bytes y cabrían $\lfloor 4092/40 \rfloor = 102$ entradas en vez de 170. La metapágina guarda la raíz y la cabeza de la free list.

En esta sección h es la altura del árbol y N su número de nodos, cada uno en una página. Cuando un nodo se desborda tiene $M + 1$ entradas, y el split las reparte sin dejar que un grupo pase de $\lfloor (M + 1)/2 \rfloor + 1$, así que el otro recibe al menos $\lfloor M/2 \rfloor$. Una inserción solo agrega entradas o divide nodos llenos, así que nunca deja un nodo con menos de $\lfloor M/2 \rfloor$ entradas. Ese mínimo solo se pierde con los borrados, porque la eliminación es lazy y no rebalancea los nodos que quedan poco llenos, y la carga masiva STR deja casi todos los nodos llenos. La altura solo crece cuando se divide la raíz, y ni los borrados ni la carga masiva la aumentan, así que con n puntos insertados es $\mathcal{O}(\log_M n)$.

Hoja del R-Tree

cabecera 4 B	(x, y) 16 B	RID 8 B	(x, y)	RID	...	(x, y)	RID
-----------------	------------------	------------	----------	-----	-----	----------	-----

170 entradas de 24 B

Nodo interno del R-Tree

cabecera 4 B	L 16 B	U 16 B	hijo 4 B	...	L	U	hijo
-----------------	-------------	-------------	-------------	-----	-----	-----	------

113 entradas de 36 B

Figura 19. Páginas de 4 KB de una hoja y de un nodo interno del R-Tree.

Métricas. El motor calcula la distancia entre dos puntos en metros con dos métricas, la geodésica y la euclidiana. Para dos puntos con latitudes φ_1, φ_2 y longitudes λ_1, λ_2 , en radianes, y $R_T = 6371$ km el radio de la Tierra, la geodésica es la fórmula de Haversine,

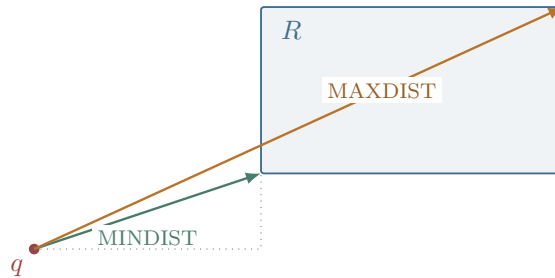
$$d = 2R_T \arcsin \sqrt{\sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}},$$

y la euclidiana proyecta los grados a metros sobre un plano local y aplica Pitágoras,

$$d = R_T \sqrt{(\Delta\lambda \cos \bar{\varphi})^2 + \Delta\varphi^2},$$

con $\bar{\varphi}$ la latitud media. A escala de una ciudad las dos dan casi lo mismo, y la euclidiana se aleja al crecer la distancia.

Para recorrer el árbol, cada métrica se extiende a un punto q y un MBR R de dos formas (figura 20). MINDIST(q, R) es una cota inferior de la distancia de q a cualquier punto de R , y MAXDIST(q, R) es una cota superior. Si R es un punto, las dos coinciden con la distancia exacta.

**Figura 20.** MINDIST y MAXDIST entre un punto y un MBR.

Consultas por rango. Hay tres casos según la forma del rango (figura 21). Una consulta por radio (q, r) solo baja por los hijos con $\text{MINDIST}(q, R) \leq r$, porque los demás no pueden tener puntos dentro del radio. En las hojas calcula la distancia de cada punto a q , y los que están a r o menos se agregan al resultado. Si un hijo tiene $\text{MAXDIST}(q, R) \leq r$, todos los puntos de su subárbol están dentro del radio, así que se agregan al resultado sin calcular sus distancias.

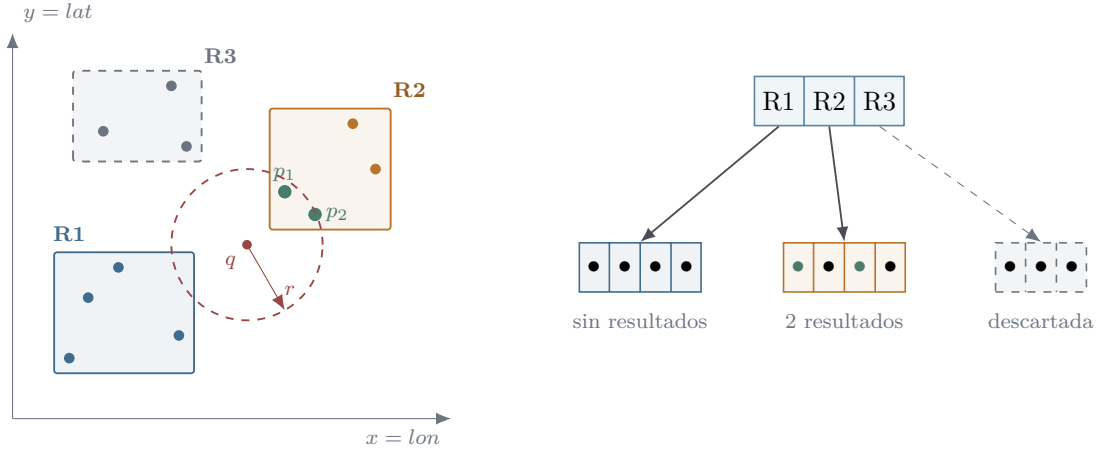


Figura 21. R-Tree con tres hojas y consulta por radio (q, r) .

Una consulta por rectángulo (L, U) usa la misma idea con la geometría de las cajas. Un MBR intersecta el rectángulo si se solapan en los dos ejes, es decir, $x_{min} \leq U_x$ y $L_x \leq x_{max}$, y lo mismo en y , y está contenido en él si $L_x \leq x_{min}$ y $x_{max} \leq U_x$, y lo mismo en y . La consulta solo baja por los hijos cuyo MBR intersecta el rectángulo, y en las hojas agrega al resultado los puntos que caen dentro. Si el MBR de un hijo está contenido en el rectángulo, todos los puntos de su subárbol se agregan al resultado sin revisarlos uno por uno.

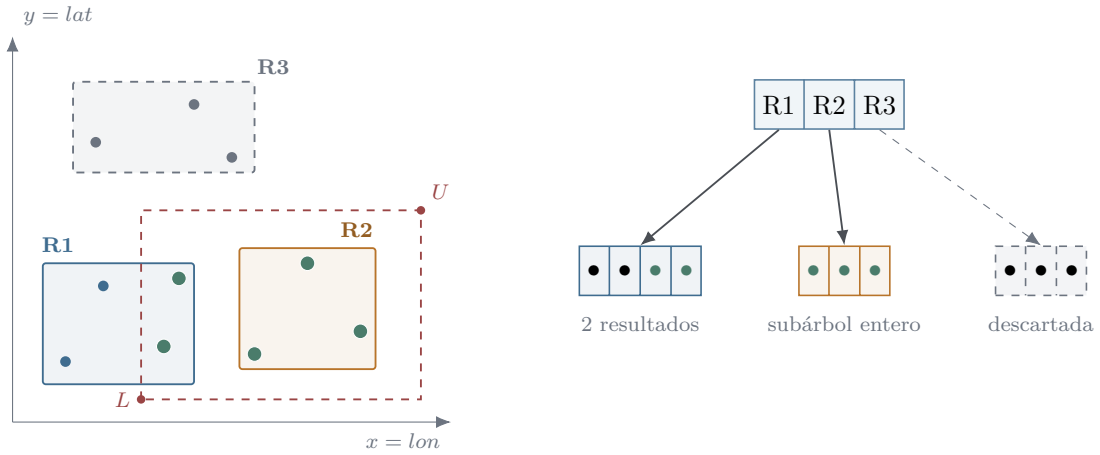


Figura 22. R-Tree con tres hojas y consulta por rectángulo (L, U) .

La búsqueda exacta de un punto p es el caso $L = U = p$. Baja por los hijos cuyo MBR contiene a p y en las hojas agrega al resultado las entradas con ese mismo punto. Si en cada nivel un solo MBR contiene a p , el costo es $\mathcal{O}(h)$, y si varios lo contienen hay que revisar cada uno de sus subárboles, con $\mathcal{O}(N)$ en el peor caso.

Una consulta por polígono (figura 23) recorre el árbol igual que una por rectángulo, usando el MBR del polígono. En las hojas, cada punto que cae dentro de ese MBR pasa por un ray casting, que lanza un rayo horizontal desde el punto y cuenta cuántos lados del polígono cruza (figura 24). Por el teorema de la curva de Jordan, el punto está dentro si el número de cruces es impar, y en ese caso se agrega al resultado. Un punto sobre el borde cuenta como dentro. Si el MBR de un hijo está entero dentro del polígono, todos los puntos de

su subárbol se agregan al resultado sin revisarlos uno por uno.

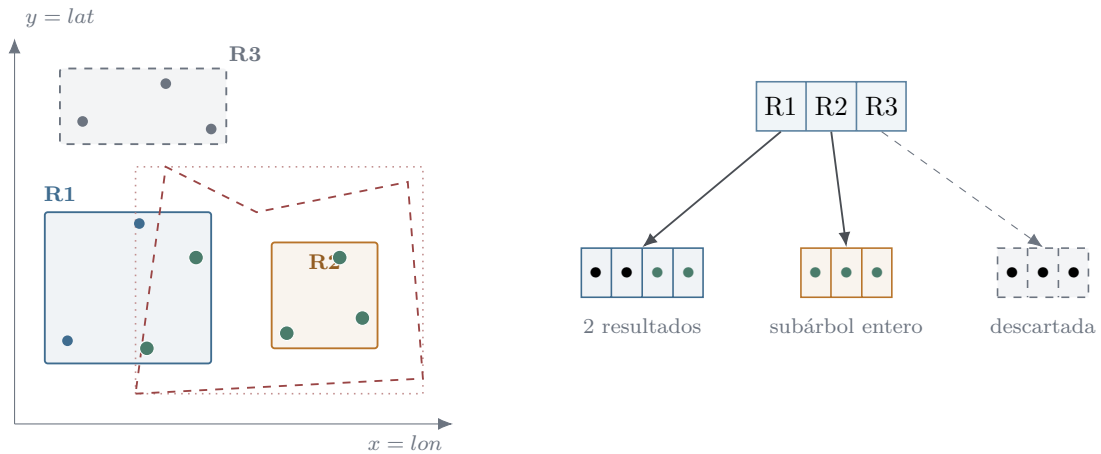


Figura 23. R-Tree con tres hojas y consulta por polígono.

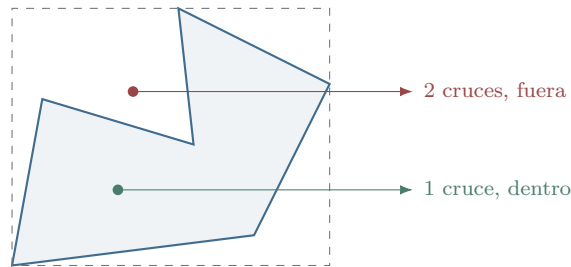


Figura 24. Ray casting dentro del MBR del polígono.

En los tres casos, los MBR de los hijos de un mismo nodo pueden solaparse, así que el área de la consulta puede tocar varios de ellos y hay que bajar por todos, aunque algunos no tengan resultados. Con poco solapamiento se visitan cerca de $h + K$ nodos, con K las hojas con resultados, y leer las k filas cuesta a lo sumo B I/O ([apartado 8.4.1](#)). En el peor caso se visita el árbol entero, así que el costo es $\mathcal{O}(N + B)$.

k-NN. Los k vecinos más cercanos se buscan con un recorrido best-first ([listado 1](#) y [figura 25](#)). Una sola cola de prioridad, ordenada por MINDIST, guarda nodos y puntos, y en cada paso se saca el primero. Si es un nodo, se leen sus entradas y se agregan a la cola, y si es un punto, es el siguiente vecino, porque cualquier punto más cercano estaría dentro de un nodo con menor MINDIST, que habría salido antes. Así los vecinos salen en orden de distancia, y los nodos con MINDIST mayor que la distancia al k -ésimo vecino nunca llegan a leerse. La cola se mantiene en memoria, así que el costo es el de los nodos leídos más 1 I/O por vecino, y en el peor caso $\mathcal{O}(N + k)$.

```

1 kNN(q, k):
2   C ← cola de prioridad con (0, raíz)
3   V ← lista vacía
4   mientras C no está vacía y |V| < k:
5     (d, e) ← sacar el mínimo de C
6     si e es un punto:
7       agregar e a V

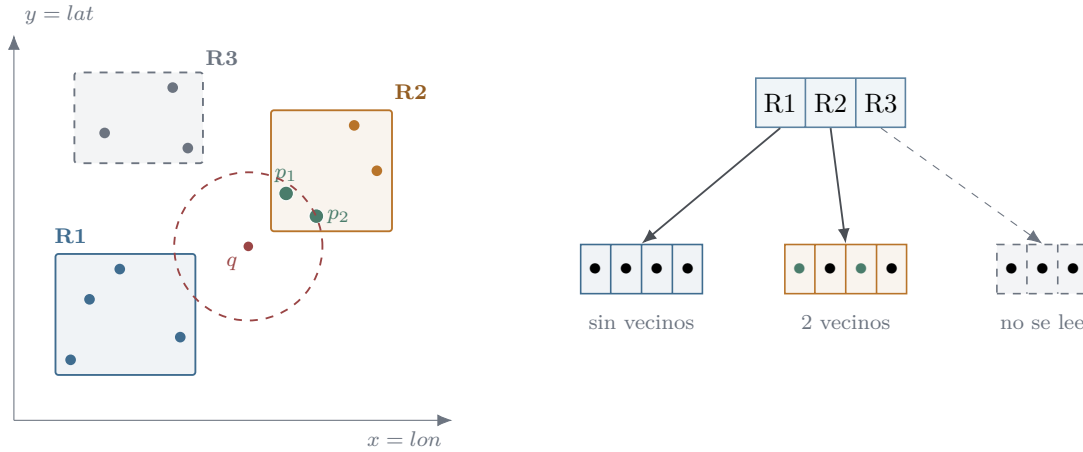
```

```

8      sino:
9          leer el nodo e
10         para cada entrada f de e:
11             insertar (MINDIST(q, f), f) en C
12     devolver V

```

Listing 1. k-NN best-first.

Figura 25. k-NN con $k = 2$ sobre el árbol de la figura 21.

Inserción. Entre el MBR de un hijo y la entrada nueva se define una preferencia que compara primero el MINDIST, después el agrandamiento del MBR y por último su área. Aquí el MINDIST es entre dos MBR y se mide en grados sobre el plano (x, y) , porque solo se usa para comparar. Para insertar se baja desde la raíz, eligiendo en cada nivel el hijo de menor preferencia. Si el punto queda fuera de todos los hijos decide el MINDIST, y si queda dentro de varios, todos empatan en MINDIST y agrandamiento cero, así que va al de menor área. En la hoja se agrega la entrada y se actualizan los MBR del camino hacia la raíz mientras cambien.

Si un nodo queda con $M + 1$ entradas, se divide en dos. Las semillas son el par de entradas más lejano según MINDIST. El resto se recorre de la más cercana a alguna semilla a la más lejana, y cada entrada va al grupo de menor preferencia, hasta que un grupo llega a la mitad más uno y lo que queda va al otro. Recorrer en ese orden hace que cada grupo crezca desde su semilla hacia el centro, y el desempate por área evita que una caja grande siga creciendo, así que los dos nodos quedan con poco solapamiento. El padre recibe la entrada del nodo nuevo y se divide con la misma lógica si se llena, y si se divide la raíz el árbol crece un nivel. El costo es h lecturas, una escritura por nivel cuyo MBR cambia y en el peor caso un split por nivel, $\mathcal{O}(h)$.

Eliminación. La eliminación es lazy, como en el GiST de PostgreSQL. Se busca la hoja con el par $(punto, RID)$, bajando por todos los hijos cuyo MBR contiene el punto. Se quita la entrada, se recalculan los MBR del camino para que sigan siendo exactos, y un nodo solo se libera si queda vacío. Si la raíz queda con un solo hijo, ese hijo pasa a ser la raíz. No hay mínimo de ocupación ni reinserción de entradas, así que un borrado no

mueve entradas entre nodos, a cambio de que tras muchos borrados queden nodos poco llenos, que se corrigen reconstruyendo el árbol con la carga masiva. Hay dos casos según cuántos MBR contienen el punto. Si cae dentro de uno solo por nivel, el costo es $\mathcal{O}(h)$, y si varios hermanos lo contienen hay que revisar cada uno de sus subárboles, con $\mathcal{O}(N)$ en el peor caso.

Carga masiva STR. Para construir el árbol sobre una tabla existente se usa STR (*Sort-Tile-Recursive*), que lo arma de abajo hacia arriba en vez de insertar punto por punto. Se usa en `CREATE INDEX`, al reconstruir los índices después de reorganizar un Sequential File y al abrir el motor si el archivo del índice está vacío. Insertar punto por punto deja nodos llenos a medias, porque cada split los parte por la mitad, y los MBR terminan solapándose según el orden en que llegaron los puntos. STR, en cambio, ve todos los puntos desde el inicio y puede agruparlos por cercanía.

Con n entradas y M entradas por nodo hacen falta $P = \lceil n/M \rceil$ nodos. La idea es repartirlos en una grilla de unas $\sqrt{P} \times \sqrt{P}$ celdas, para que cada nodo cubra una zona casi cuadrada del plano, porque así los MBR quedan pequeños y se solapan poco. Las entradas se ordenan en memoria por x y se cortan en $S = \lceil \sqrt{P} \rceil$ franjas verticales de $S \cdot M$ entradas. Cada franja se ordena por y y se corta cada M entradas, y cada grupo es un nodo. Todos los nodos quedan llenos, salvo el último de cada franja, que recibe las entradas que sobran. En la [figura 26](#), con $n = 27$ y $M = 3$, $P = 9$ y $S = 3$, así que salen 3 franjas de 9 puntos con 3 hojas cada una. Con los MBR de esos nodos se repite lo mismo para el nivel de arriba, ordenando por el centro de cada MBR y con el M de los nodos internos, hasta que un nivel tiene un solo nodo, que es la raíz.

El árbol se arma en un archivo temporal que al final reemplaza al original, así que si el proceso muere a la mitad el índice anterior queda intacto. Leer la tabla para obtener los pares (*punto*, *RID*) cuesta B , y cada página del árbol se escribe una sola vez, así que el costo es $\mathcal{O}(B + N)$. Después de la carga, las inserciones y eliminaciones siguen usando los algoritmos anteriores.

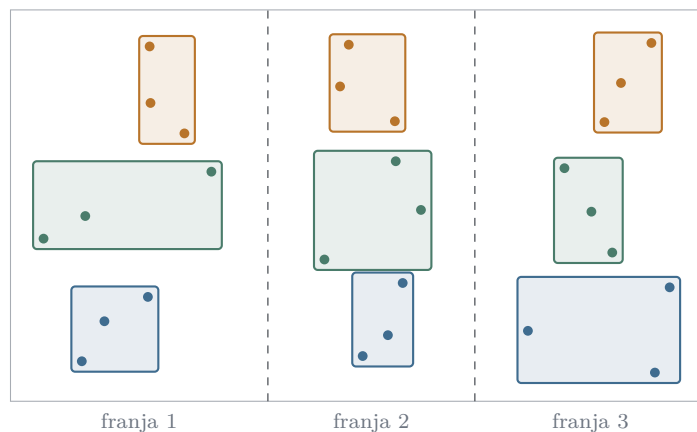


Figura 26. Hojas de una carga masiva STR con $n = 27$ y $M = 3$.

7. Algoritmos externos

Los algoritmos externos resuelven ORDER BY, GROUP BY y JOIN acotando su memoria de trabajo a un presupuesto fijo de B páginas de buffer. En esta sección B denota ese presupuesto, como en clase [4], y T el número de páginas de la entrada. El sistema usa $B = 64$, es decir 256 KB, y mide el presupuesto en bytes, porque los registros con VARCHAR no tienen un tamaño fijo.

Los runs y las particiones se escriben en archivos temporales propios de cada consulta, que se borran al terminar aunque la consulta falle. Estos archivos no pasan por el buffer pool, porque se escriben y se leen una sola vez, de forma secuencial, y no tendría sentido que compitan con las páginas de las tablas y los índices.

Alcance: Las rutinas de `engine/external.py` reciben las filas como un iterador y limitan su memoria de trabajo a B páginas. En un JOIN entre dos tablas, el ejecutor les entrega directamente el recorrido de cada una. En el ORDER BY y el GROUP BY de un SELECT, en cambio, las filas que produce el acceso (recorrido, índice o bitmap) ya están en una lista en memoria, de modo que el presupuesto acota la memoria adicional del algoritmo y no la de la consulta completa.

Cuándo se usan: Un ORDER BY no ordena si el acceso ya entrega las filas en ese orden, como el recorrido de un B+ Tree o de un Sequential File por su clave, y un JOIN usa un join anidado con índice si alguna de las dos columnas tiene uno. El GROUP BY, en cambio, siempre usa external hash, porque no existe una ruta alternativa. Una agregación sin GROUP BY, como COUNT(*), se calcula en una sola pasada con un acumulador en memoria y no usa estos algoritmos.

7.1 External sort

Fase de runs: Se leen filas hasta que su tamaño empaquetado supera $B \cdot 4096$ bytes, se ordenan en memoria y se escriben a disco como un run ordenado. Se repite hasta consumir la entrada, lo que deja $\lceil \text{bytes} / (B \cdot 4096) \rceil$ runs, es decir, unos $\lceil T/B \rceil$ cuando las páginas de entrada están llenas. Si toda la entrada cabe en el presupuesto, se ordena en memoria y no se escribe nada.

Fase de merge: Cada pasada mezcla grupos de hasta $B - 1$ runs. La mezcla mantiene un heap con la fila actual de cada run, y en cada paso saca la menor y avanza ese run. El modelo de $B - 1$ páginas de entrada y una de salida fija el número de runs por pasada, pero el código lee los runs en streaming y no administra páginas de buffer individuales. Las pasadas se repiten mientras queden más de $B - 1$ runs, y la última no se escribe, sino que entrega las filas directamente al resto de la consulta. Un ORDER BY descendente invierte la comparación de la clave.

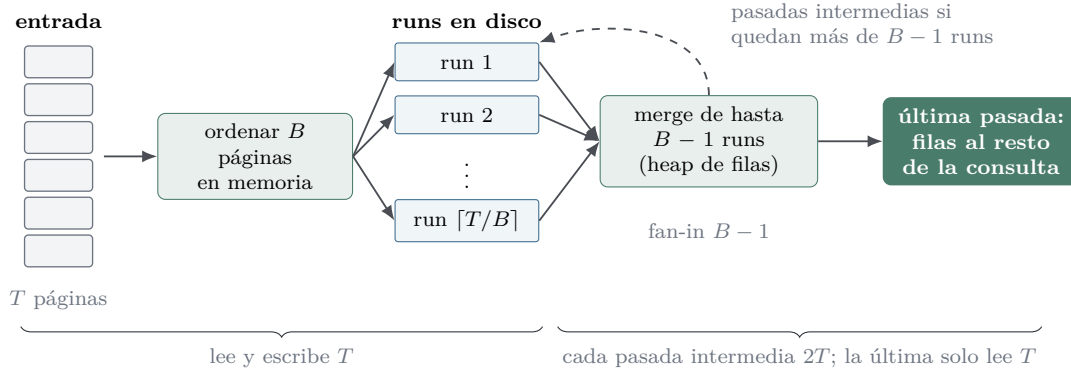


Figura 27. External sort con B páginas de buffer.

Costo: Con $m = \lceil \log_{B-1} \lceil T/B \rceil \rceil$ pasadas de merge, la fase de runs lee y escribe T páginas, cada pasada intermedia lee y escribe T , y la última solo lee T , así que el costo es

$$2T + 2T(m - 1) + T = 2Tm + T.$$

En el plan de ejecución, **runs** es el número de runs escritos y **pasadas** es m , que incluye la pasada final aunque no se escriba. Con 0 runs, 0 pasadas el ordenamiento ocurrió en memoria. Por ejemplo, en una corrida con $B = 4$ y 20 000 registros de **estudiantes** se generaron 57 runs y $m = 4$ pasadas, porque $\lceil \log_3 57 \rceil = 4$.

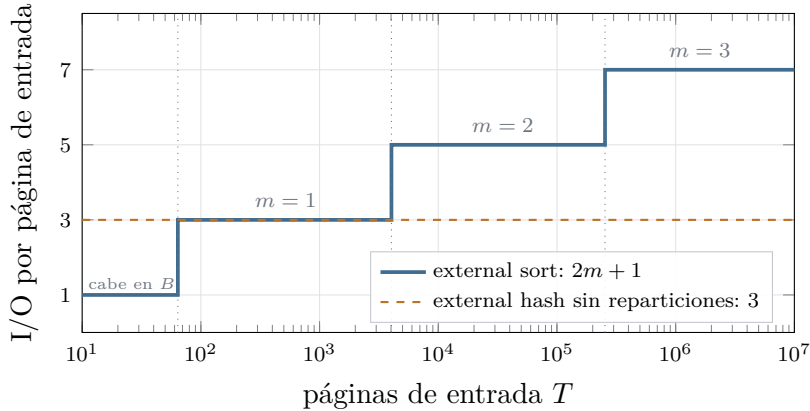


Figura 28. Costo del external sort con $B = 64$. Los saltos ocurren en $T = B$, $B(B - 1) = 4032$ y $B(B - 1)^2 = 254016$ páginas. El external hash cuesta 3 desde la primera página, porque no tiene el atajo de memoria del sort.

7.2 External hash

GROUP BY y JOIN comparten la misma idea. En la fase de particionamiento, una función de hash h_p reparte las filas en $B - 1$ particiones en disco según su clave, de modo que todas las filas con la misma clave caen en la misma partición. En la fase siguiente cada partición se procesa por separado en memoria, con una tabla hash que usa una función distinta de h_p .

Si una partición no cabe en las B páginas, se vuelve a particionar con otra función de hash, que incluye el nivel como parte de la clave. Esto se repite como máximo 4 veces (**MAX_REPARTITION**), porque si todas las filas de una partición tienen la misma clave ningún

hash las separa, y entonces se procesa en memoria igual. En una prueba con $B = 2$ y todas las filas con la misma clave, el algoritmo hizo exactamente 4 reparticiones antes de procesar.

Diferencia con el sort: El external sort evita el disco cuando la entrada cabe en B páginas. El GROUP BY y el hash join no tienen ese atajo: siempre escriben las particiones, y por eso su costo es $3T$ y $3(T_R + T_S)$ desde la primera página. Con 8 filas, un GROUP BY igual produce 63 particiones. Sus resultados salen en el orden de las particiones, por lo que un ORDER BY posterior se aplica después, con external sort.

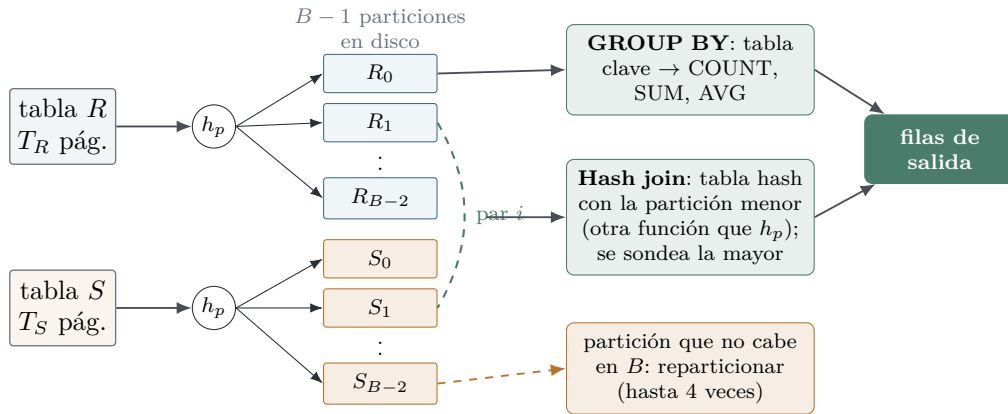


Figura 29. External hash: particionamiento con h_p y procesamiento por partición para GROUP BY y hash join.

7.2.1 GROUP BY

Cada partición se lee de forma secuencial y se acumula en una tabla hash que asocia cada clave con sus agregados, ya sea COUNT, SUM o AVG. Como cada grupo está completo en una sola partición, al terminar cada una sus grupos ya son definitivos. Sin embargo, el resultado de todos los grupos se conserva en un solo diccionario hasta el final, de modo que la memoria usada crece con el número de grupos distintos y no con el tamaño de la entrada. El presupuesto B limita la lectura de cada partición y las reparticiones, no ese diccionario. Sin reparticiones, el costo es leer la entrada, escribir las particiones y leerlas, es decir $3T$.

7.2.2 Hash join

Las dos tablas se particionan con la misma h_p sobre la columna del join, así que una fila de la partición i de una tabla solo puede coincidir con filas de la partición i de la otra. Para cada par de particiones, si alguna está vacía se descarta, y si no, se construye una tabla hash con la menor y se recorre la mayor buscando coincidencias. Solo hace falta que la partición menor quepa en las B páginas. Si T_R y T_S son las páginas de las dos tablas, el costo sin reparticiones es $3(T_R + T_S)$.

El hash join se usa solo cuando ninguna de las dos columnas del JOIN tiene índice. Si la tabla de la derecha tiene índice, el ejecutor recorre la de la izquierda y busca cada valor en ese índice. Si no, prueba con el índice de la tabla de la izquierda, y recién después recorre

al hash join. Cada búsqueda lee sus filas con un bitmap de RIDs ([apartado 8.4.1](#)). Con varios JOIN, el resultado acumulado de los anteriores entra como un lado del siguiente. El resultado del JOIN se materializa en una lista antes de continuar con el WHERE, el GROUP BY y el ORDER BY.

8. Procesamiento de consultas SQL

Una sentencia SQL pasa cuatro etapas principales dentro del módulo `query/`. Primero, el lexer divide la consulta en tokens; luego, el parser utiliza estos tokens para construir el árbol de sintaxis abstracta (AST). Después, el analizador semántico verifica que la consulta sea válida según la información almacenada en el catálogo. Finalmente, la clase `Conexion`, implementada en `query/conexion.py`, se encarga de coordinar la ejecución de la consulta. En esta etapa, se solicitan los locks necesarios, se determina la ruta de acceso más adecuada y se ejecuta la operación correspondiente.

Si ocurre algún problema durante este proceso, el sistema identifica la etapa en la que se produjo el error y devuelve una respuesta específica. Los errores pueden clasificarse como `lexico`, `sintactico`, `semantico`, `ejecucion`, `transaccion`, `deadlock` o `timeout`.

8.1 Lexer, parser y AST

El lexer (`query/lexer.py`) recorre el texto carácter por carácter. Una palabra se busca en la tabla de *keywords* sin distinguir mayúsculas, y si no está, es un identificador, que sí distingue mayúsculas. Los números aceptan un punto decimal, los strings van entre comillas simples y los operadores de dos caracteres (`<=`, `>=`, `!=` y `<>`) se reconocen antes que los de uno.

El parser (`query/parser.py`) es descendente recursivo, con un método por sentencia. Las condiciones del WHERE se analizan por niveles de precedencia, de menor a mayor OR, AND, NOT y comparación, y los paréntesis vuelven al nivel más bajo. Algunas construcciones se reescriben al armar el árbol: `x BETWEEN a AND b` se convierte en `x >= a AND x <= b`, `NOT LIKE`, `NOT IN` y `NOT BETWEEN` se envuelven en un nodo `NotCondition`, y una llamada a función dentro del WHERE produce una `SpatialCondition`. El AST está formado por las clases de `query/ast.py`, una por tipo de sentencia ([figura 30](#)).

Cuadro 3. Cobertura SQL implementada.

Familia	Sintaxis disponible
DDL	CREATE TABLE <i>t</i> (...) [USING HEAP SEQUENTIAL], con exactamente una PRIMARY KEY; DROP TABLE <i>t</i> ; CREATE INDEX [<i>nombre</i>] ON <i>t</i> (<i>col</i>) [USING BTREE BPLUS HASH RTREE], con B+ por defecto.
DML	INSERT INTO <i>t</i> VALUES (...) de una fila con todas las columnas; UPDATE <i>t</i> SET <i>c</i> = <i>v</i> , ... [WHERE]; DELETE FROM <i>t</i> [WHERE].

Familia	Sintaxis disponible
Consulta	SELECT con *, columnas, agregados o funciones espaciales con AS; FROM t [JOIN ...]; WHERE; GROUP BY de una columna; ORDER BY de una columna o de una función de distancia, ASC o DESC; LIMIT n.
Predicados	=, ≠ (!= o <>), <, ≤, >, ≥, BETWEEN, [NOT] LIKE con % y _, [NOT] IN con lista o subconsulta, AND, OR, NOT, paréntesis y comparación entre dos columnas.
Agregados	COUNT(*), COUNT(col), SUM(col) y AVG(col).
Join	INNER equi-join con JOIN t2 ON t1.a = t2.b, encadenable, con columnas calificadas por su tabla.
Transacción	BEGIN [TRANSACTION] o START TRANSACTION; COMMIT o END [TRANSACTION]; ROLLBACK o ABORT.
Diagnóstico	EXPLAIN y EXPLAIN ANALYZE sobre SELECT, INSERT, UPDATE y DELETE.
Espacial	POINT(lat, lon), POLYGON(POINT(...), ...), DISTANCIA, DISTANCIA_GEO, DISTANCIA_HAVERSINE, DISTANCIA_EUCLIDIANA, DENTRO_DE y SET var = POINT(...).

No se soportan NULL, literales TRUE/FALSE, DISTINCT, HAVING, OFFSET, MIN, MAX, varias claves de ORDER BY o GROUP BY, alias de tabla, outer joins, aritmética, INSERT con lista de columnas o con varias filas, ALTER TABLE ni DROP INDEX. Tampoco se pueden combinar agregados, ORDER BY espacial ni subconsultas IN con un JOIN. Una comilla simple no puede escaparse dentro de un string.

```

1 -- estudiantes(id INT PRIMARY KEY, nombre VARCHAR(40),
2 -- carrera VARCHAR(30), promedio FLOAT), Heap
3 SELECT carrera, COUNT(*)
4 FROM estudiantes
5 GROUP BY carrera
6 ORDER BY carrera;

```

Listing 2. Consulta relacional sobre la tabla de demostración `estudiantes`.

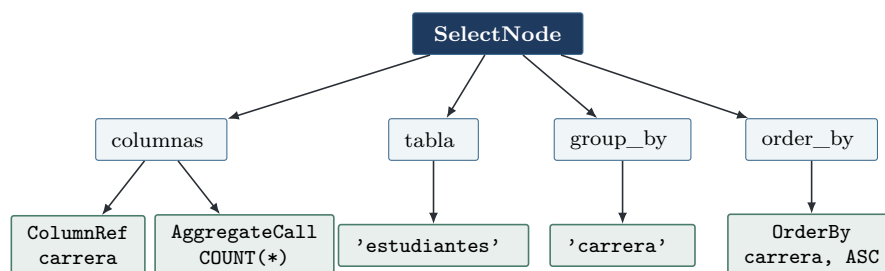


Figura 30. AST del SELECT del listado 2, con las clases de `query/ast.py`.

```

1 CREATE TABLE matriculas (

```

```
2  id INT PRIMARY KEY,
3  alumno INT,
4  curso INT
5 ) USING HEAP;
6
7 CREATE INDEX idx_curso ON matriculas(curso) USING HASH;
8 INSERT INTO matriculas VALUES (1, 3, 201);
9
10 SELECT estudiantes.nombre, cursos.titulo
11 FROM matriculas
12 JOIN estudiantes ON matriculas.alumno = estudiantes.id
13 JOIN cursos ON matriculas.curso = cursos.codigo;
```

Listing 3. DDL, DML y JOIN con las tablas de demostración.

8.2 Análisis semántico

query/semantic.py recorre el AST antes de ejecutar y rechaza lo que no tiene sentido para el esquema. Las reglas que aplica son:

- La tabla y cada columna referenciada deben existir. En un JOIN, las columnas se califican con su tabla, el ON compara una columna de cada lado con tipos compatibles y una tabla no se une dos veces.
- En el WHERE, el valor comparado debe ser compatible con el tipo de la columna. Una columna POINT solo se usa dentro de una función espacial, LIKE solo se aplica a CHAR o VARCHAR, y una subconsulta de IN debe devolver una sola columna de tipo compatible.
- SUM y AVG necesitan una columna numérica y solo COUNT admite *. Con agregados y GROUP BY, la única columna suelta permitida es la de agrupación, y sin GROUP BY no se pueden mezclar columnas y agregados. Las funciones espaciales del SELECT no se combinan con agregados.
- INSERT debe traer un valor por columna y de su tipo. UPDATE no puede repetir una columna en el SET ni modificar la clave primaria.
- CREATE TABLE exige nombres de columna distintos, tipos válidos y una sola PRIMARY KEY. CREATE INDEX con RTREE solo se acepta sobre POINT, y BTREE o HASH no se aceptan sobre POINT.
- LIMIT debe ser un entero positivo.

Algunas restricciones dependen del índice y se detectan al ejecutar. Por ejemplo, un hash sobre FLOAT falla al construirse, porque `index/hash_utils.py` rechaza claves flotantes ([apartado 6.1](#)).

8.3 Catálogo

`query/catalog.py` guarda, por tabla, su esquema, el objeto de almacenamiento (Heap o Sequential), la columna clave y un diccionario de índices que asocia cada columna con su objeto y su tipo (`bplus`, `clustered`, `hash` o `rtree`). Por eso cada columna admite un solo índice.

Al crear una tabla, el motor le agrega un índice sobre la clave primaria: un B+ no agrupado y único en una tabla Heap, y el B+ agrupado en una tabla Sequential. Las tablas e índices creados por SQL se guardan en `data/catalog.json` mediante `query/catalog_store.py`, que escribe un archivo temporal y lo reemplaza con `os.replace`. Al arrancar se vuelven a abrir, y un índice se reconstruye solo si su archivo está vacío. Los archivos se nombran como `tabla_columna_tipo.idx`, así que el nombre escrito en CREATE INDEX se analiza pero no se usa. El catálogo guarda un número de versión que no se valida al cargar, y dos DDL concurrentes sobre tablas distintas pueden sobreponerse al reescribir el archivo, porque los locks son por tabla y no por catálogo.

8.4 Planificación y ejecución

El motor no mantiene estadísticas ni estima selectividad. Un SELECT se resuelve con reglas que se revisan en orden, y la primera que aplica fija la ruta de acceso ([figura 31](#)):

1. Si hay JOIN, cada join usa un join anidado con índice cuando alguna de las dos columnas tiene uno, y un external hash join en otro caso. Con varios JOIN se encadenan sobre las filas acumuladas, y el WHERE se aplica después.
2. Si el WHERE o el ORDER BY tienen un patrón espacial compatible y la columna tiene R-Tree, se usa el índice ([apartado 8.5](#)).
3. Sin WHERE, un ORDER BY sobre una columna con B+ no agrupado recorre el índice, y uno sobre la clave de un Sequential File sigue su lista enlazada. En otro caso se recorre la tabla completa.
4. Una igualdad simple sobre una columna indexada usa ese índice, ya sea B+ agrupado, B+ no agrupado o hash. Con los dos últimos, las filas se leen a través de un bitmap de RIDs ([apartado 8.4.1](#)).
5. Una igualdad sobre la clave de un Sequential File sin índice usa la búsqueda binaria en MAIN y luego recorre AUX.
6. Si el WHERE es una combinación con AND u OR de condiciones que los índices pueden resolver, se arma un bitmap por cada una, se combinan con AND u OR y se leen las páginas resultantes una sola vez ([apartado 8.4.1](#)). Un rango sobre el B+ agrupado no entra en esta regla.
7. Si de las conjunciones del WHERE sale un rango sobre una columna con B+, agrupado o no, se usa ese rango y el resto del WHERE se evalúa sobre los candidatos.
8. En cualquier otro caso se recorre la tabla y se filtra fila por fila.

Con las filas obtenidas, `_finalizar_select` aplica en orden el GROUP BY con external hash o la agregación global, el ORDER BY con external sort si la ruta no dejó las filas ordenadas, la proyección, las funciones espaciales del SELECT y el LIMIT.

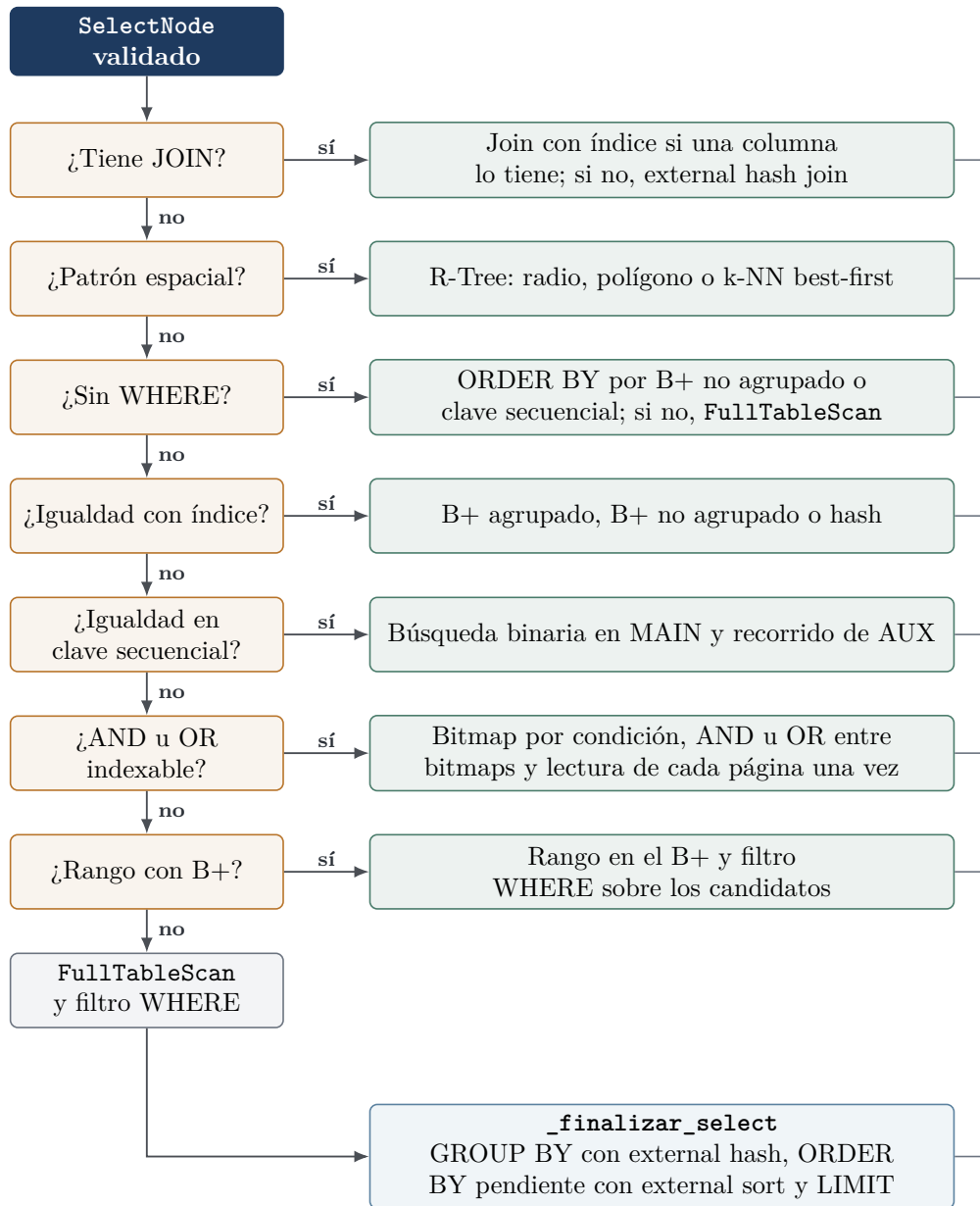


Figura 31. Orden en que `ejecutar_select` de `query/conexion.py` elige la ruta de acceso. GROUP BY y el external sort se aplican después sobre las filas obtenidas.

Una subconsulta de IN se ejecuta primero y su resultado se guarda como lista, así que la pertenencia de cada fila cuesta $\mathcal{O}(M)$, y la consulta completa $\mathcal{O}(NM)$.

8.4.1 Bitmap index scan y bitmap heap scan

Los índices no agrupados devuelven RIDs. En lugar de leer una fila por cada RID, el ejecutor los agrupa en un `RidBitmap`, que asocia cada página con el conjunto de sus slots (`common/bitmap.py`). Dos bitmaps se combinan con AND y con OR página por página, y la tabla se lee recorriendo las páginas del bitmap en orden físico, cada una una sola vez.

En un Sequential File se leen primero las páginas de MAIN y luego las de AUX, que se identifican con su página negativa. Si P es el número de páginas distintas con resultados, leer las filas cuesta P I/O, que es a lo sumo el número de RIDs y a lo sumo B . Después de leer, el WHERE completo se evalúa de nuevo sobre cada fila, de modo que el bitmap solo reduce el conjunto de candidatos.

Un bitmap se construye a partir de las condiciones que un índice sí resuelve: igualdad o IN con hash o B+ no agrupado, comparaciones de rango con B+ no agrupado, y radio o polígono con R-Tree. Dentro de un AND, los términos que no se pueden resolver con índices se omiten y quedan para la verificación final, y los rangos de una misma columna se juntan en un solo recorrido del B+. Un OR solo genera bitmap si todas sus ramas se pueden resolver con índices, porque una rama sin índice obligaría a recorrer la tabla. NOT no genera bitmap. Si ninguna condición es indexable, la consulta sigue por la regla siguiente.

INSERT, DELETE y UPDATE trabajan así:

- **INSERT.** En una tabla Heap se verifica que la clave primaria no exista buscándola en su índice. En una tabla Sequential la inserción pasa por el B+ agrupado, que también rechaza duplicados. Después se insertan las entradas en todos los índices no agrupados.
- **DELETE.** Una igualdad sobre la clave de una tabla Sequential borra por el B+ agrupado. Cualquier otra condición recorre la tabla, y en una tabla Heap el borrado se hace por RID. Las entradas se quitan de todos los índices no agrupados.
- **UPDATE.** Se implementa como un DELETE seguido de un INSERT de la fila modificada, así que la fila puede cambiar de RID, y no se permite modificar la clave primaria.

Después de cada INSERT, DELETE o UPDATE sobre una tabla Sequential se revisa la política de reorganización ([apartado 5.2](#)), y si se dispara, los índices no agrupados se reconstruyen con bulk load, porque cambian los RIDs. En autocommit no hay undo por sentencia, así que un error entre escribir la fila y actualizar todos sus índices puede dejar un efecto parcial. Dentro de una transacción explícita ese efecto se deshace con ROLLBACK ([apartado 9](#)).

COUNT cuenta todas las filas, porque no existe NULL, y AVG sobre una entrada vacía devuelve 0. Un GROUP BY sin agregados devuelve la columna de agrupación y una columna `count`, aunque no se haya pedido.

8.5 Extensión espacial

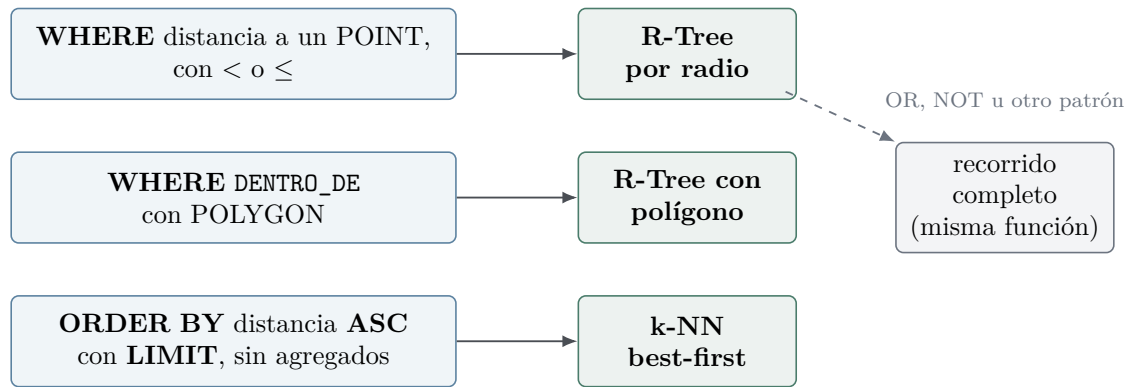
El lexer, el parser y el analizador semántico reconocen POINT, POLYGON y las funciones espaciales. DISTANCIA, DISTANCIA_GEO y DISTANCIA_HAVERSINE usan la fórmula de Haversine, DISTANCIA_EUCLIDIANA usa la proyección local ([apartado 6.5](#)) y DENTRO_DE(`col`, POLYGON(...)) es un predicado booleano. Con SET `mi_ubicacion` = POINT(...) se guarda un punto por sesión, que el ejecutor sustituye en las funciones antes de validar la consulta, y una función de distancia en el SELECT puede llevar un alias con AS.

```
1 CREATE TABLE tiendas (  
2   id INT PRIMARY KEY,  
3   nombre VARCHAR(40),  
4   categoria VARCHAR(20),  
5   ubicacion POINT  
6 ) USING HEAP;  
7  
8 CREATE INDEX idx_ubicacion  
9 ON tiendas(ubicacion) USING RTREE;  
10  
11 -- Radio de 5 km alrededor de UTEC.  
12 SELECT id, nombre,  
13         DISTANCIA(ubicacion, POINT(-12.1354, -77.0224)) AS d  
14 FROM tiendas  
15 WHERE DISTANCIA(ubicacion, POINT(-12.1354, -77.0224)) <= 5000;  
16  
17 -- Cinco vecinos mas cercanos con un punto de sesion.  
18 SET mi_ubicacion = POINT(-12.1354, -77.0224);  
19 SELECT id, nombre  
20 FROM tiendas  
21 ORDER BY DISTANCIA(ubicacion, mi_ubicacion) ASC  
22 LIMIT 5;  
23  
24 -- Tiendas dentro de un poligono.  
25 SELECT id FROM tiendas  
26 WHERE DENTRO_DE(ubicacion, POLYGON(POINT(-12.13, -77.04),  
27   POINT(-12.13, -77.01), POINT(-12.16, -77.01),  
28   POINT(-12.16, -77.04))));
```

Listing 4. Consultas espaciales.

El ejecutor usa el R-Tree solo con tres patrones (figura 32). Una consulta por radio necesita una función de distancia entre la columna POINT y un punto literal, comparada con $<$ o $\leq$. Una consulta por polígono necesita DENTRO_DE(col, POLYGON(...)). Un k-NN necesita un ORDER BY ascendente por distancia con LIMIT, sin GROUP BY ni agregados. Si el WHERE es una conjunción, el patrón puede ser uno de sus términos, y los demás se evalúan sobre los candidatos que devuelve el índice. En el k-NN, ese filtro se aplica a medida que el recorrido best-first entrega vecinos, hasta reunir k filas que lo cumplen. OR, NOT y cualquier otro patrón recorren la tabla y evalúan la misma función fila por fila.

Como el índice y el recorrido completo evalúan la misma función, se puede comprobar que devuelven los mismos IDs. CREATE INDEX recorre la tabla y construye el R-Tree con STR, y INSERT, DELETE y UPDATE lo mantienen como a cualquier otro índice.



Los filtros adicionales unidos con AND se reevalúan sobre los candidatos.

Figura 32. Patrones SQL que activan el R-Tree.

8.6 Plan de ejecución (EXPLAIN)

Mientras ejecuta, **Conexion** agrega a una lista una línea por cada paso, por ejemplo búsqueda por índice hash en 'carrera=...' o ORDER BY carrera ASC (external sort, B=64: 0 runs, 0 pasadas). Los pasos de external sort, external hash y join incluyen las estadísticas reales de runs, pasadas, particiones y reparticiones, y los del R-Tree, los nodos visitados y los candidatos.

EXPLAIN y EXPLAIN ANALYZE toman los mismos locks que la sentencia. EXPLAIN ejecuta un SELECT, pero en INSERT, UPDATE y DELETE solo cuenta las filas que resultarían afectadas, sin escribir en disco. EXPLAIN ANALYZE ejecuta la sentencia y agrega el tiempo real y las filas obtenidas.

EngineAdapter convierte la lista en el árbol que muestra la interfaz (figura 33). Cada paso se clasifica por sus palabras clave en un nodo como FullTableScan, IndexScan (Extendible Hash), SequentialBinarySearch, Sort (ORDER BY), Aggregate (GROUP BY) o, en el camino de bitmaps, Bitmap Index Scan (con el índice usado), BitmapAnd, BitmapOr y Bitmap Heap Scan, cada nodo queda como padre del anterior y la raíz es un nodo Projection. El campo execution_time_ms mide el tiempo real de la consulta, mientras que cost, estimated_time_ms y rows_estimated son valores fijos por tipo de nodo o tomados del resultado, de modo que no son un modelo de costos comparable al de PostgreSQL.

El plan cambia según la ruta elegida. Sobre **estudiantes**, que tiene hash en **carrera** y ningún índice en **nombre**:

Consulta	Paso del plan	Nodo
WHERE carrera = 'Bioingenieria'	busqueda por indice hash	IndexScan (Extendible Hash)
WHERE nombre = 'Ana Torres'	escaneo completo + filtro WHERE	FullTableScan

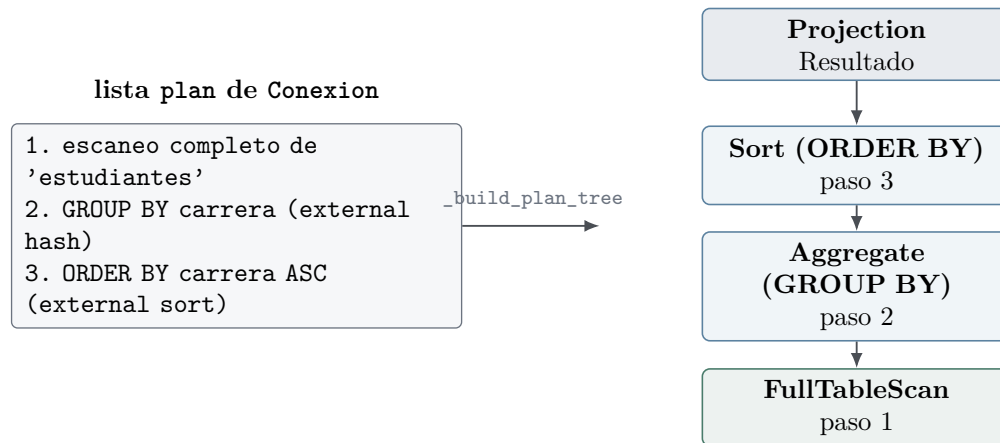


Figura 33. Conversión del plan textual al árbol del PlanPanel: cada paso se vuelve un nodo, el primero queda como hoja y **Projection** es la raíz.

9. Transacciones y concurrencia

Cada sentencia llega con un `session_id`. Si la sesión tiene una transacción abierta, la sentencia forma parte de ella, y si no, se ejecuta en autocommit. En los dos casos, **Conexion** toma los locks de todas las tablas que la sentencia lee o escribe, incluidas las de los JOIN y de las subconsultas, antes de ejecutarla. Un SELECT pide locks compartidos (S) y las demás sentencias, exclusivos (X).

9.1 Lock manager

`transaction/locks.py` guarda, por tabla, el conjunto de sesiones con lock S y la sesión con lock X, si la hay. La compatibilidad es la clásica (cuadro 4). Una sesión que ya tiene un lock no lo vuelve a pedir, y una que tiene S y pide X lo convierte en X cuando es la única lectora.

Cuadro 4. Matriz de compatibilidad.

Solicitado / existente	S	X
S	compatible	espera
X	espera	espera

Si el lock no se puede otorgar, la sesión espera en una condition variable, que se revisa cada 0,2 segundos y cada vez que otra sesión libera un lock. Antes de cada espera, el lock manager registra en el grafo wait-for a quiénes espera la sesión y busca, con un recorrido en profundidad desde ella, un ciclo que vuelva a ella. Si lo encuentra hay un deadlock, y la sesión que lo detectó es la víctima. Si no lo hay y pasan tres segundos, la espera termina por timeout. Si la sesión estaba dentro de una transacción, **Conexion** la aborta con un ROLLBACK completo; en autocommit solo falla la sentencia y se liberan sus locks. En ambos casos devuelve el error `deadlock` o `timeout`.

En una transacción explícita los locks se retienen hasta COMMIT o ROLLBACK, y como no se libera ninguno antes, tampoco los compartidos, el protocolo equivale a un 2PL riguroso (un 2PL estricto que además retiene los locks S). En autocommit se liberan

al terminar la sentencia. La granularidad es la tabla completa, y no hay prioridad por antigüedad ni cola justa entre las sesiones que esperan. Como los locks se retienen hasta el final de la transacción, el protocolo evita los tres problemas de concurrencia vistos en clase [5]. No hay lectura sucia, porque un `SELECT` espera el lock `S` mientras otra transacción tenga `X` sobre la tabla y solo lee después de su `COMMIT` o `ROLLBACK`. No hay lectura no repetible, porque el lock `S` de la primera lectura impide que otra transacción escriba hasta que la lectora termine. Tampoco hay actualización perdida, pero con un costo: si dos transacciones leen el mismo dato y luego lo actualizan, ambas tienen `S` y ambas piden `X`, cada una espera a la otra y el lock manager aborta a una por deadlock. El sistema no implementa el bloqueo de actualización (protocolo `PU`), que evitaría ese caso haciendo esperar a la segunda desde la lectura.

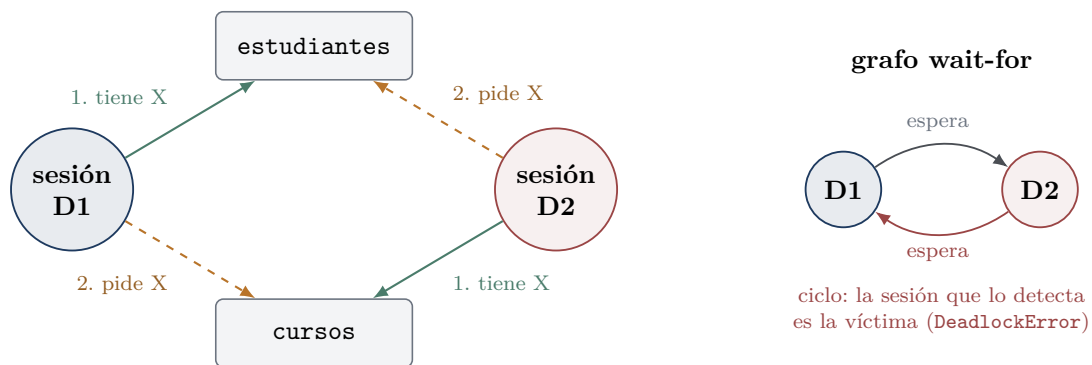


Figura 34. Escenario `escenario_deadlock` de `transaction/simulation.py`: dos sesiones insertan en **estudiantes** y **cursos** en orden cruzado. El grafo wait-for se indexa por sesión.

9.2 Transaction manager

`transaction/manager.py` mantiene una transacción activa por sesión. Cada `BEGIN` crea una transacción con un ID correlativo (`T1001`, `T1002`, ...) y una lista de undo vacía. Mientras la sesión ejecuta una sentencia, su transacción queda asociada al hilo, y el `FileManager` la consulta antes de cada escritura (apartado 4):

- antes de sobrescribir una página guarda una entrada `WRITE` con su versión anterior, leída del archivo;
- al agregar una página guarda una entrada `APPEND` con su número;
- antes de truncar un archivo a menos páginas, o de reemplazarlo, guarda una entrada `SNAPSHOT` con su contenido completo.

`COMMIT` descarta la lista y libera los locks. `ROLLBACK` cierra en el buffer pool los archivos afectados para descartar sus frames, recorre la lista en orden inverso restaurando las páginas, truncando lo agregado y reescribiendo los snapshots, libera los locks y recarga el estado en memoria de todas las tablas e índices, como el mapa de espacio libre del Heap o la raíz de los árboles. `CREATE TABLE`, `DROP TABLE` y `CREATE INDEX` no se permiten dentro de una transacción.

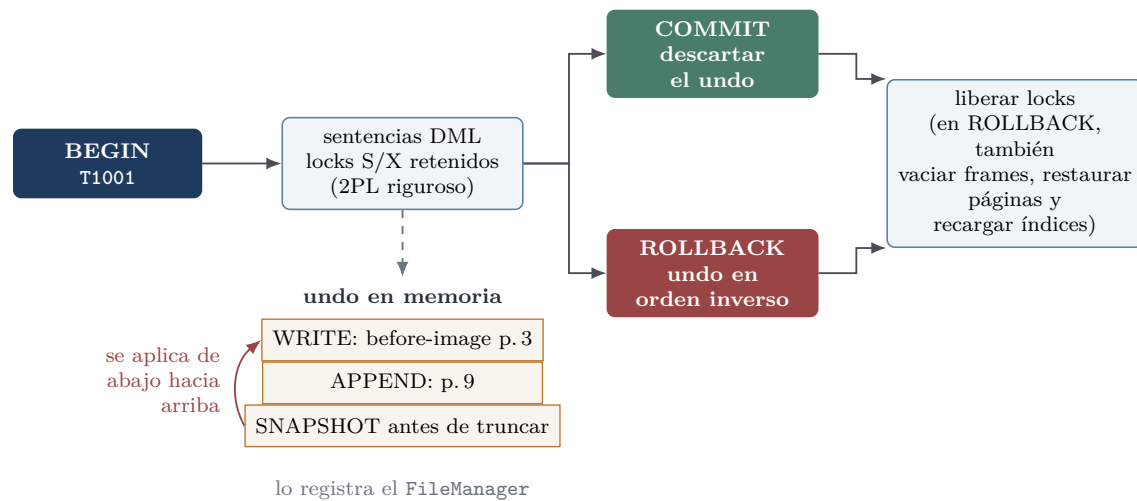


Figura 35. Ciclo de vida de una transacción explícita y su lista de undo.

No es recuperación ante caídas

No existe WAL, redo, checkpoint ni ARIES. Las páginas se escriben y hacen `flush()` antes del commit; el undo desaparece si cae el proceso y no se invoca `fsync`. Por tanto, el sistema ofrece rollback durante la ejecución, pero no atomicidad/durabilidad garantizadas frente a caída de proceso, sistema operativo o energía [5].

No hay timeout por inactividad, así que una sesión que abre una transacción y no la cierra retiene sus locks. El frontend guarda el `session_id` en `localStorage`, por lo que dos pestañas del mismo navegador comparten sesión. Los endpoints auxiliares de puntos, visualización, seed y reorganización no toman locks.

9.3 Serializabilidad

El lock manager guarda la historia de locks otorgados, con sesión, tabla, modo e instante. `transaction/serializability.py` construye con ella un grafo de precedencia, con una arista de T_i a T_j cuando T_i accedió antes a la misma tabla y al menos uno de los dos accesos fue X. Si el grafo no tiene ciclos, un orden topológico da el orden serial equivalente. La historia también incluye los locks de las transacciones abortadas, así que la víctima de un deadlock aparece en el grafo. Es una verificación posterior a la ejecución, que usa la simulación, no el mecanismo que controla la concurrencia.

9.4 Simulación con hilos

`transaction/simulation.py` crea una base temporal y lanza hilos que se sincronizan con una barrera para empezar a la vez. Tiene dos escenarios:

- **Carrera.** Varios hilos insertan en `estudiantes` claves primarias que se repiten entre ellos. En modo `locked` cada hilo usa `BEGIN`, `INSERT` y `END TRANSACTION` a través de `Conexion`. En modo `raceless` cada hilo comprueba si la clave existe, espera unos milisegundos para ensanchar la ventana de la carrera y escribe directamente en el Heap y los índices, sin locks.

- **Deadlock.** Dos sesiones insertan en `estudiantes` y en `cursos` en orden cruzado (figura 34).

Al terminar, la simulación verifica que no haya claves primarias duplicadas en `estudiantes` y que cada entrada del índice B+ sobre `id` apunte a una fila con su clave. En el escenario de carrera, además, verifica que toda inserción reportada como exitosa esté en disco. En modo `locked` además construye el grafo de precedencia e imprime el orden serial equivalente, y en el escenario de deadlock comprueba que una de las sesiones fue elegida como víctima. En modo `raceless`, las inconsistencias que aparecen son la evidencia de la carrera.

9.5 Resultados de la simulación

Se ejecutó la simulación con sus valores por defecto: 8 hilos, 3 claves en disputa (500, 501 y 502), 50 ms de espera en la ventana de la carrera y semilla 42. Los comandos son:

```
python transaction/simulation.py                # carrera, modo locked
python transaction/simulation.py --mode raceless # carrera, sin locks
python transaction/simulation.py --scenario deadlock # deadlock
```

Cuadro 5. Escenario de carrera con 8 hilos y 3 claves en disputa.

	locked	raceless
INSERT reportados como exitosos	3 de 8	8 de 8
Claves primarias duplicadas	0	2 (500 y 501)
Entradas del índice B+ incoherentes	0	1
INSERT exitosos que no están en disco	0	1 (clave 502)
Grafo de precedencia	acíclico	—

En modo `locked` se insertó exactamente una fila por clave. Los otros cinco hilos encontraron la clave ya insertada, recibieron el error de clave duplicada e hicieron `ROLLBACK`. Las tres verificaciones pasaron y el grafo de precedencia resultó acíclico, con el orden serial equivalente `race-7` $\rightarrow$ `race-6` $\rightarrow$ `race-1` $\rightarrow$ `race-5` $\rightarrow$ `race-0` $\rightarrow$ `race-2` $\rightarrow$ `race-3` $\rightarrow$ `race-4`. Cada nombre `race-i` es la sesión del hilo *i*, y como cada hilo abre una sola transacción, el orden entre sesiones es también el orden serial equivalente entre sus transacciones. En cinco ejecuciones más, este modo no presentó ninguna inconsistencia.

En modo `raceless` los ocho hilos reportaron éxito, porque todos comprobaron que su clave no existía antes de que los demás escribieran. El resultado tiene claves duplicadas, una entrada del índice que apunta a una fila equivocada y una inserción reportada como exitosa que no quedó en disco (inserción perdida: otro hilo sobrescribió la misma página del Heap sin haber visto esa fila). Como el intercalado de los hilos lo decide el sistema operativo, el resultado cambia en cada ejecución. En diez ejecuciones adicionales, todas presentaron al menos una inconsistencia: entre 0 y 3 claves duplicadas, entre 0 y 5 entradas del índice incoherentes y entre 0 y 2 inserciones perdidas.

```

1 [hilo-2] T(D2) BEGIN TRANSACTION; adquiere X en 'cursos'
2 [hilo-1] T(D1) BEGIN TRANSACTION; adquiere X en 'estudiantes'
3 [hilo-1] T(D1) esperando X en 'cursos' ...
4 [hilo-2] T(D2) esperando X en 'estudiantes' ...
5 [hilo-2] T(D2) deadlock: deadlock detectado (D2 -> D1 -> D2); victima=D2
6 [hilo-1] T(D1) adquiere X en 'cursos' -> END TRANSACTION (commit)
7
8 [resultado] D1: commit
9 [resultado] D2: deadlock
10 [verificacion] deadlock detectado y victima abortada: SI
11
12 [verificacion] PK unicas OK en 'estudiantes'
13 [verificacion] 0 entradas de indice incoherentes
14 [verificacion] Schedule SERIALIZABLE (grafo de precedencia aciclico)
15 [verificacion] orden serial equivalente: D2 -> D1

```

Figura 36. Salida completa del escenario de deadlock.

En el escenario de deadlock (figura 36), D2 fue la segunda sesión en esperar y detectó el ciclo $D2 \rightarrow D1 \rightarrow D2$, así que fue la víctima. Su transacción se deshizo, lo que liberó el lock X sobre cursos, y D1 pudo terminar con COMMIT. El grafo de precedencia incluye a D2 aunque haya sido abortada, porque la historia registra todos los locks otorgados.

10. API e interfaz de usuario

10.1 API REST

`backend/main.py` define la API con FastAPI y crea una sola instancia de `EngineAdapter`, que conecta `Conexion`, el catálogo, el `BufferPool`, el `FileManager`, el gestor de transacciones y el lock manager. Al arrancar registra las tablas de demostración (apartado 3) y vuelve a abrir las tablas creadas por SQL.

Cuadro 6. Contrato REST implementado.

Método	Ruta	Función
GET	/api/health	Estado del servicio.
GET	/api/session	Genera un UUID de sesión.
GET	/api/tables	Esquema, organización, índices y estadísticas de cada tabla.
POST	/api/query	Ejecuta una sentencia SQL en una sesión.
GET	/api/tables/{table}/points	Puntos de una columna POINT (submuestreados), su bounding box y si tiene R-Tree.
GET	/api/visualize/{table}	Estado físico de las estructuras de la tabla para el inspector.
POST	/api/tables/reorganize	Reorganiza una tabla Sequential y reconstruye sus índices.
POST	/api/seed	Inserta los datos de ejemplo si las tablas están vacías.

Para POST `/api/query`, `EngineAdapter` toma primero una foto de las estructuras de la

tabla si la sentencia es INSERT, DELETE o UPDATE, ejecuta la sentencia con **Conexion**, mide el tiempo y arma el árbol del plan. La respuesta incluye las columnas y filas, el plan, el estado de la transacción de la sesión, un descriptor espacial cuando la consulta es de radio, polígono o k-NN, y la información para animar los cambios en las estructuras. Un error de SQL se devuelve con HTTP 200 y **status** = `.error`, con la etapa que falló. Los demás endpoints responden HTTP 400 ante una tabla o columna inválida. CORS acepta cualquier origen, lo que sirve para desarrollo local, pero no es una configuración de producción. FastAPI genera además la documentación en `/docs` y `/redoc`.

10.2 Interfaz

La interfaz (`frontend/`) es una aplicación React con Vite (figura 37). A la izquierda está el panel de archivos, arriba el editor de consultas y abajo cuatro pestañas: Resultados, Plan de ejecución, Inspector físico y Laboratorio espacial, este último con el mapa.

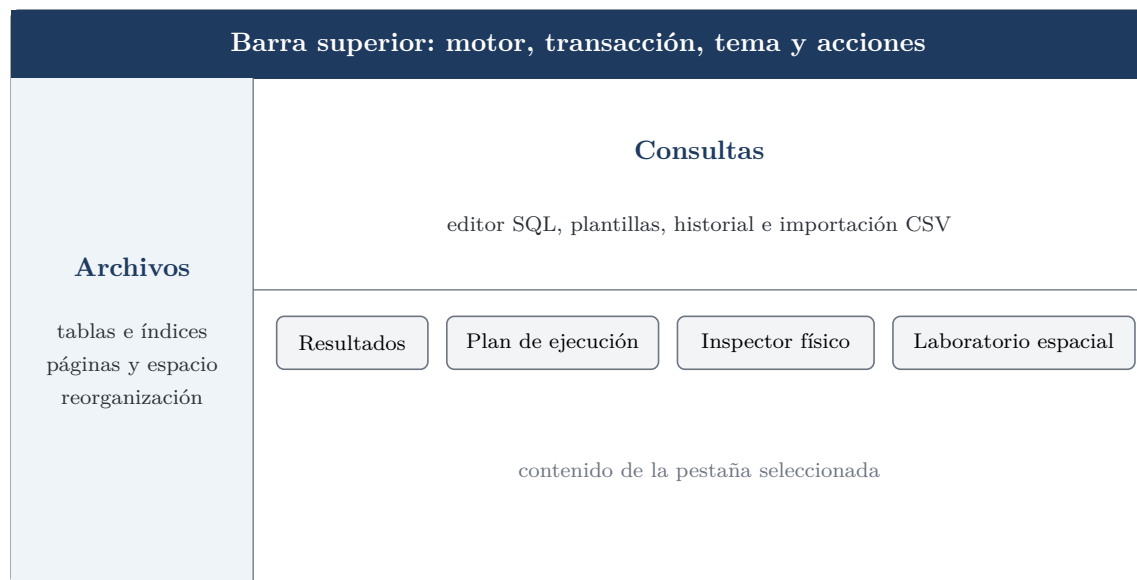


Figura 37. Distribución funcional de la interfaz.

`frontend/src/App.jsx` genera el `session_id` una vez y lo guarda en `localStorage`, junto con el tema y el alto del editor. Un script se divide por punto y coma y cada sentencia se envía por separado. Si una falla con una transacción abierta, el cliente envía ROLLBACK, y como la división es textual, un punto y coma dentro de un string también corta la sentencia. Después de ejecutar, la interfaz abre el mapa si la consulta fue espacial, el inspector si hubo cambios que animar, y los resultados en otro caso.

Panel de archivos.

Lista las tablas con sus columnas, clave primaria, organización e índices, y sus estadísticas de páginas, registros y espacio desperdiciado (figura 38a). Para las tablas Sequential indica si necesitan reorganización y permite reorganizarlas. También permite borrar una tabla, con confirmación. Las estadísticas se calculan recorriendo la tabla, así que cuestan $\mathcal{O}(N)$.

Panel de consultas.

Editor con números de línea, ejecución con Ctrl/Cmd+Enter, plantillas de consultas, historial e importación de CSV (figura 38b).

Resultados.

Tabla de filas con el tiempo de ejecución, exportación a CSV y copia del resultado en JSON, sin paginación (figura 39a).

Plan de ejecución.

Muestra el árbol del plan (figuras 33 y 39b).

Inspector físico.

Se detalla en el apartado 10.3.

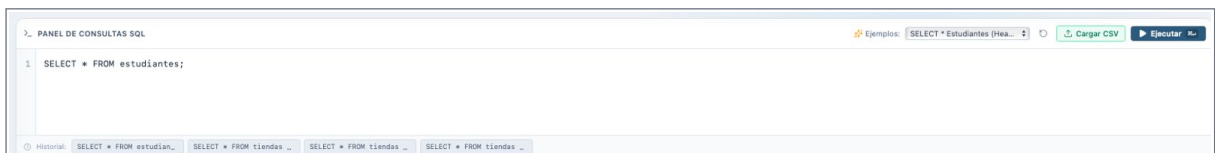
Laboratorio espacial.

Se detalla en el apartado 10.4.

Las figuras 38 y 39 muestran los cuatro paneles que pide el enunciado, tomados de la aplicación en ejecución.



(a) Panel de archivos: tablas cargadas, esquema, organización e índices.



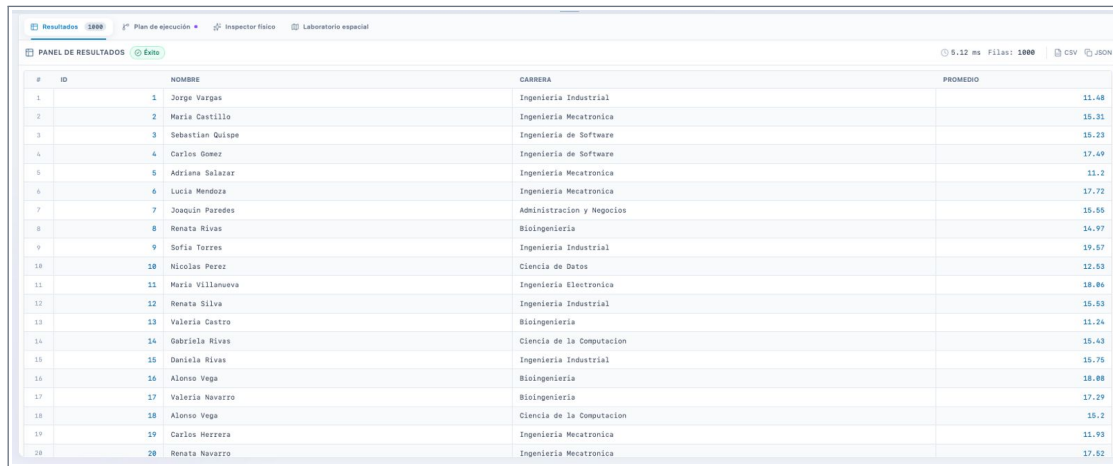
(b) Panel de consultas: editor SQL con plantillas e historial.

Figura 38. Paneles de archivos y de consultas.

10.3 Inspector físico

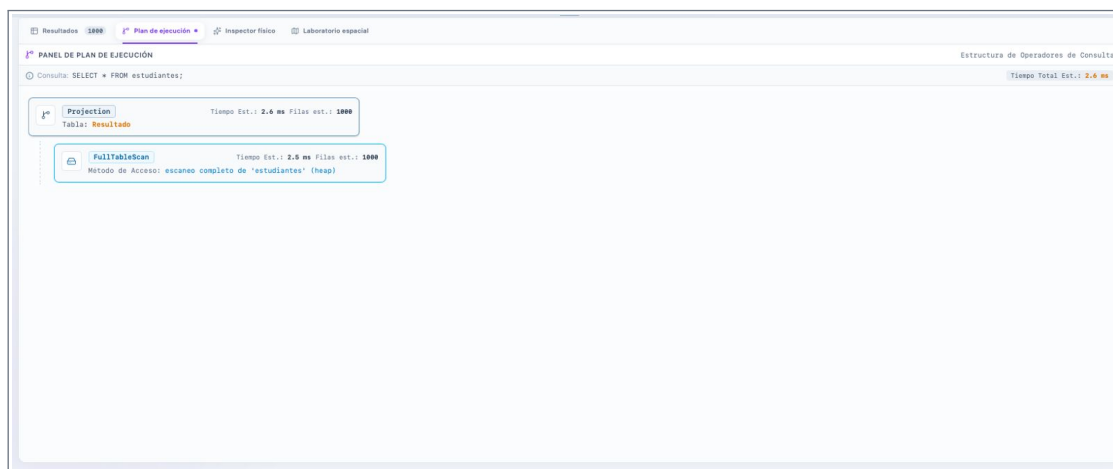
El inspector muestra cómo están guardadas las estructuras de una tabla, con una pestaña para el B+ Tree, el hash, el Sequential File y el R-Tree. Los datos salen de `backend/visualize.py`, que pide a cada estructura una foto de sus páginas: los nodos del B+ con sus claves y enlaces, el directorio y los buckets del hash con su local depth y sus cadenas de overflow, las entradas de MAIN y AUX con sus punteros, y los MBR del R-Tree.

Cuando una sentencia modifica la tabla, el backend activa antes el registro de eventos de los índices, como los splits del B+ o la duplicación del directorio del hash, y compara la foto anterior con la posterior. Como con páginas de 4 KB un split recién aparece después de cientos de inserciones, el inspector ofrece también una vista didáctica, que repite la



#	ID	NOMBRE	CARRERA	PROMEDIO
1	1	Jorge Vargas	Ingeniería Industrial	11.48
2	2	Maria Castillo	Ingeniería Mecatrónica	15.21
3	3	Sebastian Quispe	Ingeniería de Software	15.23
4	4	Carlos Gomez	Ingeniería de Software	17.49
5	5	Adriana Salazar	Ingeniería Mecatrónica	11.2
6	6	Lucia Mendoza	Ingeniería Mecatrónica	17.72
7	7	Joaquin Paredes	Administración y Negocios	15.55
8	8	Renata Rivas	Biología	14.97
9	9	Sofia Torres	Ingeniería Industrial	19.57
10	10	Nicolas Perez	Ciencia de Datos	12.53
11	11	Maria Villanueva	Ingeniería Electrónica	18.86
12	12	Renata Silva	Ingeniería Industrial	15.53
13	13	Valeria Castro	Biología	11.24
14	14	Gabriela Rivas	Ciencia de la Computación	15.43
15	15	Daniela Rivas	Ingeniería Industrial	15.75
16	16	Alonso Vega	Biología	18.88
17	17	Valeria Navarro	Biología	17.29
18	18	Alonso Vega	Ciencia de la Computación	15.2
19	19	Carlos Herrera	Ingeniería Mecatrónica	11.93
20	20	Renata Navarro	Ingeniería Mecatrónica	17.52

(a) Panel de resultados: filas devueltas y tiempo de ejecución.



(b) Panel de plan de ejecución: índice usado y orden de las operaciones.

Figura 39. Paneles de resultados y de plan de ejecución.

operación sobre una copia del algoritmo con capacidad pequeña, de cuatro hijos por nodo en el B+ y dos entradas por bucket en el hash, para que cada split, merge o duplicación se vea paso a paso. La copia sigue las mismas reglas que el índice real y solo cambia la unidad de capacidad. Una reorganización muestra MAIN y AUX antes y después.

10.4 Laboratorio espacial

`frontend/src/components/MapPanel.jsx` dibuja los puntos sobre un mapa de Leaflet con teselas de OpenStreetMap. Como fondo carga hasta 4000 puntos de la tabla con `/api/tables/{tabla}/points`, que recorre todos los puntos y luego toma una muestra uniforme, de modo que el límite reduce lo que se transfiere y no la lectura. El bounding box se calcula sobre la muestra, y si la tabla tiene varias columnas POINT se usa la primera.

El usuario elige la tabla, la métrica y el modo, que puede ser radio, k-NN, polígono o solo navegar. Un clic no llama al índice directamente, sino que genera la consulta SQL correspondiente y la envía al mismo POST `/api/query`, así que recorre todo el pipeline del motor (figura 40). Con la respuesta, el mapa resalta los puntos resultantes, dibuja el

círculo o el polígono y, en un k-NN, las líneas al centro y el orden de hasta 25 vecinos. Si la consulta usó el R-Tree, también dibuja sus MBR, con colores distintos para hojas, nodos internos, nodos visitados y nodos descartados.

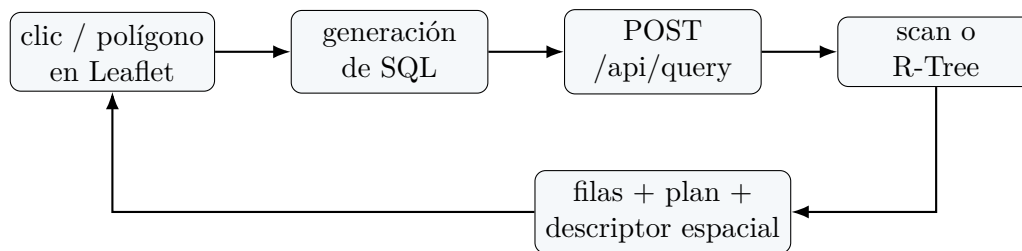


Figura 40. Flujo de una interacción espacial.

Las teselas requieren conexión a Internet. El mapa no usa PostGIS: todas las consultas pasan por el motor propio.

10.5 Importación CSV

`frontend/src/components/CsvImportModal.jsx` y `frontend/src/utils/csvImport.js` leen el archivo en el navegador, infieren para cada columna INT, FLOAT o VARCHAR con el largo máximo observado, y permiten ajustar el esquema y la organización antes de generar un script con un CREATE TABLE y un INSERT por fila. El script se coloca en el editor y se ejecuta como cualquier otro, así que cada fila es una petición separada, sin bulk load ni transacción automática, y si falla una fila, las anteriores ya quedaron insertadas. No se admiten valores vacíos como NULL ni columnas POINT, y un BIGINT grande puede perder precisión al pasar por los números de JavaScript.

11. Comparación experimental

11.1 Entorno y metodología

Las pruebas se ejecutaron en una computadora con Apple Silicon (ARM64, 8 núcleos), 16 GB de memoria y un SSD NVMe, con macOS Sequoia 15.6 y Python 3.14. Como referencia espacial se usó PostgreSQL 18.6 con PostGIS 3.6.0, al que los scripts se conectan con `psycpg2-binary`.

Las consultas sobre los índices relacionales ([cuadros 11](#) y [12](#)) se midieron después, con `data/bench_indices_consultas.py`, en otra computadora, con un AMD Ryzen 5 8645HS (12 hilos), 14 GB de memoria y un SSD NVMe, Fedora Linux 44 y Python 3.14. Sus tiempos se comparan entre sí, pero no con los de los demás cuadros.

Todas las mediciones son con caché caliente. Los datos y el índice ya se habían cargado antes de medir, y salvo en las consultas sobre los índices relacionales, los scripts no hacen una pasada de calentamiento aparte. El motor propio usa su buffer pool de 256 páginas de 4 KB (1 MB) y PostgreSQL su `shared_buffers` por defecto, de 128 MB. Esto favorece a PostgreSQL, porque su índice GiST de 100K puntos cabe entero en memoria y el R-Tree propio no cabe en el buffer pool. Los tiempos cubren solo el motor, sin red ni interfaz, pero incluyen recorrer el índice, leer las páginas y deserializar las filas devueltas (`Record.unpack` en el motor propio y `cur.fetchall()` en PostgreSQL).

Reorganización en las pruebas. Los scripts usan `SequentialFile` y `ClusteredBPlusTree` directamente, sin pasar por `Conexion`, que es la capa que dispara la reorganización automática (apartado 8.4). Por eso en las pruebas la reorganización se llama explícitamente con `reorganize()` y se mide aparte, sin mezclar su costo con el de las inserciones y eliminaciones.

Cuadro 7. Diseño del benchmark espacial.

Factor	Valores
Tamaños	1K, 10K y 100K puntos, generados de la misma forma.
Técnicas	Búsqueda secuencial, R-Tree propio y GiST de PostgreSQL.
Radio	1, 5 y 10 km.
k-NN	$k = 10, 50, 100$.
Centros	Los mismos 100 centros y la misma semilla en las tres técnicas.
Repeticiones	100 consultas por combinación, sin pasada de calentamiento.
Métricas	Tiempo de construcción, tiempo promedio de consulta, memoria y espacio en disco.
Correctitud	Las tres técnicas reciben los mismos puntos, centros y radios; los resultados no se comparan automáticamente entre ellas.

Reproducción. Los experimentos se ejecutan desde la raíz del repositorio:

```
python data/benchmark_charts.py --plot # Heap vs. Sequential y construcción
python data/bench_indices_consultas.py # consultas sobre los índices
python data/bench_churn_indices.py    # carga mixta de mutaciones
python data/bench_spatial.py --plot  # secuencial, R-Tree y GiST
```

Cada script fija sus semillas (7 y 11 en `benchmark_charts.py` y `bench_indices_consultas.py`, 42 en `bench_churn_indices.py`, y $2026 + n$ para las tiendas y 99 para los centros en `bench_spatial.py`), y los gráficos se guardan en `data/charts/`. El benchmark espacial se conecta a PostgreSQL con PostGIS usando un archivo `.env` en la raíz del repositorio o las opciones `-pg-host`, `-pg-user` y `-pg-db`, y con `-skip-pg` mide solo la búsqueda secuencial y el R-Tree.

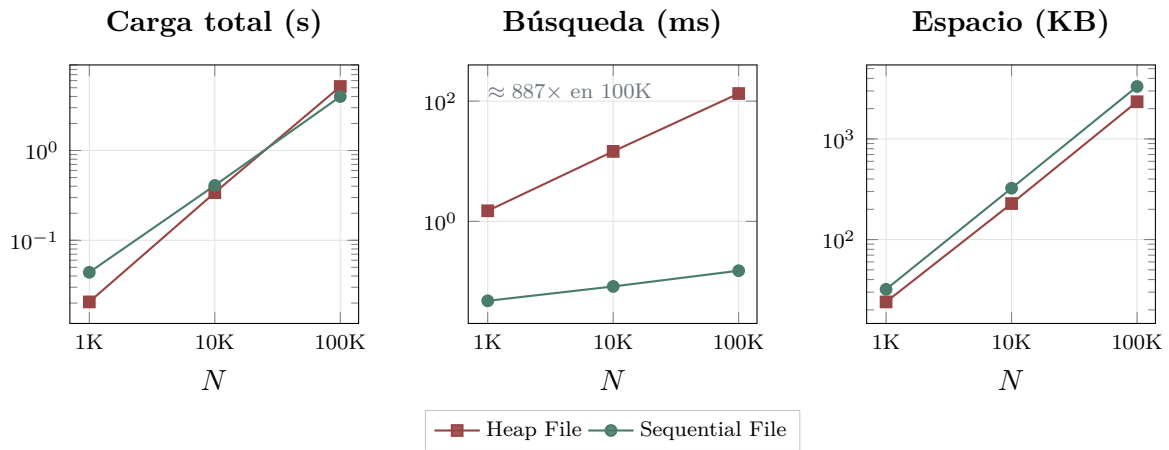
11.2 Heap File frente a Sequential File

Se insertaron registros `id+VARCHAR` en tablas de 1K, 10K y 100K filas, con claves en orden ascendente. Para cada tamaño se midió el tiempo de carga, el espacio en disco y el promedio de 30 búsquedas por clave primaria que sí existen. Es una sola ejecución por tamaño. La búsqueda en el Heap File recorre el archivo sin usar índice, y la del Sequential File baja por su B+ agrupado hasta la página de MAIN y luego recorre AUX (apartado 5.2).

Inserción. La carga ascendente favorece al Sequential File, porque cada registro va al final de MAIN y nunca pasa por AUX. Aun así, el Heap File fue más rápido en 1K (0,0206 frente a 0,0440s) y en 10K (0,339 frente a 0,408s). En 100K el orden se invierte (5,16

Cuadro 8. Heap File frente a Sequential File; una ejecución, 30 búsquedas.

N	Heap ins. (s)	Seq. ins. (s)	Heap búsq. (ms)	Seq. búsq. (ms)	Bytes H / S
1 000	0,0206	0,0440	1,497	0,0476	24 576 / 32 798
10 000	0,3393	0,4083	14,580	0,0825	233 472 / 331 806
100 000	5,1638	3,9912	133,779	0,1508	2 400 256 / 3 403 806

**Figura 41.** Datos del [cuadro 8](#).

frente a 3,99 s): de 10K a 100K el tiempo del Heap crece 15 veces y el del Sequential 9,8 veces. Hay dos causas.

- **El archivo deja de caber en el buffer pool.** Con 1K (24 KB) y 10K (233 KB) el archivo entra entero en el buffer pool de 1 MB, así que toda página que la inserción lee ya está en memoria. Con 100K (2,4 MB) no cabe, y las páginas candidatas que revisa la inserción pueden haber sido desalojadas, así que su lectura vuelve al disco. Las escrituras no explican la diferencia: en las dos organizaciones cada inserción escribe su página en el momento ([apartado 4](#)) y desalojar un frame no cuesta I/O.
- **El mapa de espacio libre crece.** Con cientos de páginas, mantener y ordenar las páginas candidatas en Python suma tiempo a cada inserción.

El Sequential File, en cambio, siempre escribe en la última página de MAIN, que se mantiene en el buffer pool, y no consulta ningún mapa de espacio libre.

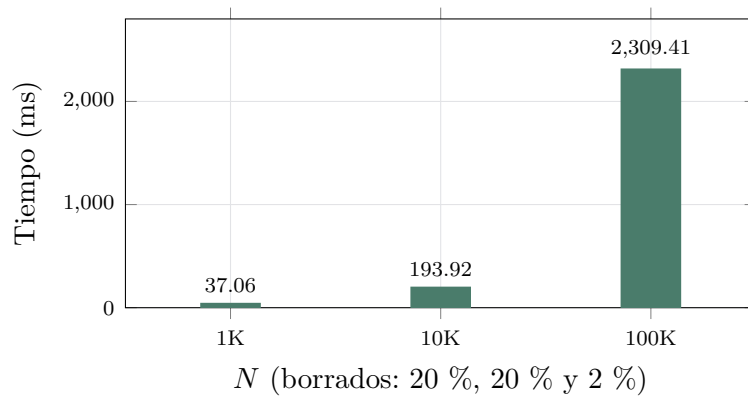
Búsqueda y espacio. En 100K la búsqueda por clave en el Sequential File (0,15 ms) fue unas 887 veces más rápida que recorrer el Heap File sin índice (133,8 ms), y ocupó un 41,8 % más de disco ([figura 41](#)). La comparación es entre un archivo sin índice y uno con su índice agrupado. En el motor, toda tabla Heap tiene además un B+ único sobre su clave primaria ([apartado 6.2](#)), con el que una búsqueda puntual tarda unos 0,2 ms ([cuadro 11](#), medido sobre otra tabla y en otra computadora).

Cuadro 9. Tiempo de reorganización del Sequential File con registros borrados.

N	Páginas MAIN	Páginas AUX	Borrados (%)	Tiempo (ms)	ms por página
1 000	12	2	20,0	37,06	2,65
10 000	124	14	20,0	193,92	1,41
100 000	1 348	54	2,0	2 309,41	1,65

Nota: ms por página = tiempo / (páginas MAIN + páginas AUX).

Reorganización. En 100K se borró solo el 2 % de las filas, porque borrar 20 000 filas una por una hacía demasiado lento el script de preparación. Como el porcentaje de borrados cambia entre tamaños (20, 20 y 2 %), la comparación entre filas del cuadro es aproximada. Desde 10K el costo por página se mantiene parecido (1,41 y 1,65 ms), como se espera de un costo $\mathcal{O}(B + A)$ (apartado 5.2); en 1K es mayor (2,65 ms) porque pesan más los costos fijos. Ninguna de las tres mediciones superaba el 30 % de borrados, y AUX tenía más páginas que el umbral $\lfloor \log_2 B \rfloor$ (14 con $B = 124$ y 54 con $B = 1 348$, frente a 6 y 10), porque el script no reorganiza solo y se llamó a `reorganize()` después de cargar.

**Figura 42.** Tiempo de reorganización del Sequential File (datos del cuadro 9).

Mientras no se reorganiza, cada página de AUX suma una lectura a la búsqueda por clave, que pasa de $\mathcal{O}(\log B)$ a $\mathcal{O}(\log B + A)$. La reorganización recorre la lista en orden de clave, descarta los registros borrados, reescribe MAIN con las páginas llenas, vacía AUX y reconstruye el B+ agrupado. El Heap File no necesita este mantenimiento, porque reutiliza el espacio de los registros borrados con su mapa de espacio libre (apartado 5.1).

11.3 B+ agrupado, B+ no agrupado y hash extendible

Comparabilidad. Las tres estructuras no se midieron en las mismas condiciones. El B+ agrupado indexa una tabla Sequential y los otros dos indexan tablas Heap, así que en rango y orden se mezcla el efecto del índice con el de la organización del archivo. En las consultas, las tres estructuras devuelven las filas completas, como el motor. El B+ agrupado lee las páginas de MAIN, y el B+ no agrupado y el hash leen sus filas con el bitmap de RIDs (apartado 8.4.1). La igualdad se midió por clave primaria en las tres, y el comportamiento con duplicados aparte, sobre `carrera`, que tiene 8 valores distintos

(cuadro 12). Antes de medir, el script comprueba que las tres estructuras devuelvan las mismas filas.

Cuadro 10. Construcción y espacio de los índices relacionales.

N	B+ agrupado		B+ no agrupado		Hash extendible	
	ms	KB	ms	KB	ms	KB
1 000	2,70	8,0	126,16	40,0	64,75	32,0
10 000	25,77	8,0	1 447,73	360,0	1 017,11	168,0
100 000	261,11	28,0	18 151,74	3 480,0	10 992,47	1 587,2

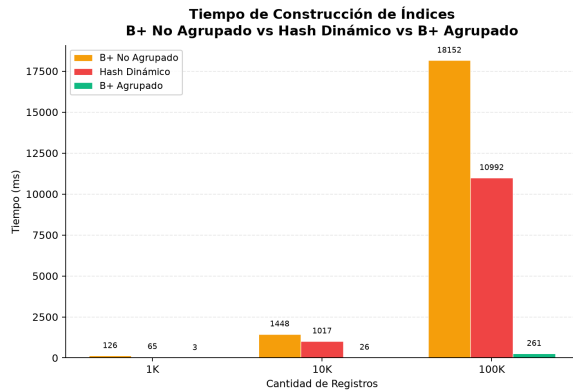


Figura 43. Tiempo de construcción de los índices.

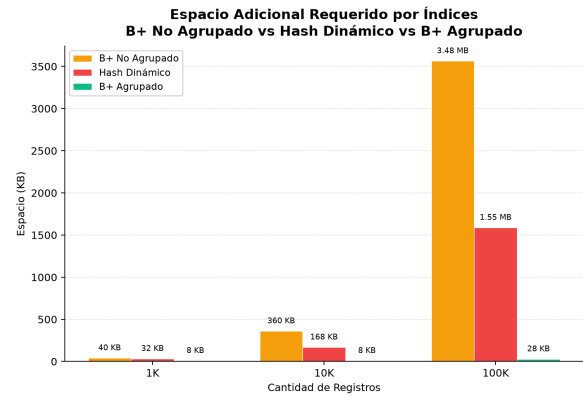


Figura 44. Espacio en disco de cada índice.

Cuadro 11. Tiempo promedio de consulta en ms. Todas las mediciones devuelven las filas completas.

N	Consulta	B+ agrupado	B+ no agrupado	Hash extendible
1 000	Igualdad (PK)	0,17	0,10	0,09
	Rango 1 %	0,22	0,16	3,55*
	Rango 50 %	2,44	2,59	3,54*
	ORDER BY	6,15 [†]	5,88	6,40*
10 000	Igualdad (PK)	0,21	0,13	0,08
	Rango 1 %	0,72	0,58	35,13*
	Rango 50 %	25,86	27,83	37,75*
	ORDER BY	59,41 [†]	58,64	153,29*
100 000	Igualdad (PK)	0,25	0,20	0,11
	Rango 1 %	4,98	4,96	354,46*
	Rango 50 %	309,19	277,76	393,74*
	ORDER BY	655,98 [†]	634,94	1 555,42*

hash no resuelve rangos ni orden. En rango se recorre la tabla Heap y se filtra, y en ORDER BY se recorre y se ordena con external sort ($B = 64$), que en 100K escribe 18 runs y hace una pasada de merge.

[†]En ORDER BY la tabla Sequential sigue su lista enlazada y el B+ agrupado no interviene.

De los cuadros 10 a 12 se observa lo siguiente.

Cuadro 12. Igualdad sobre carrera, que tiene 8 valores, en ms.

N	Filas devueltas	B+ no agrupado	Hash	Sin índice
1 000	125	0,73	0,67	3,49
10 000	1 250	6,78	6,96	35,04
100 000	12 500	72,35	79,07	359,55

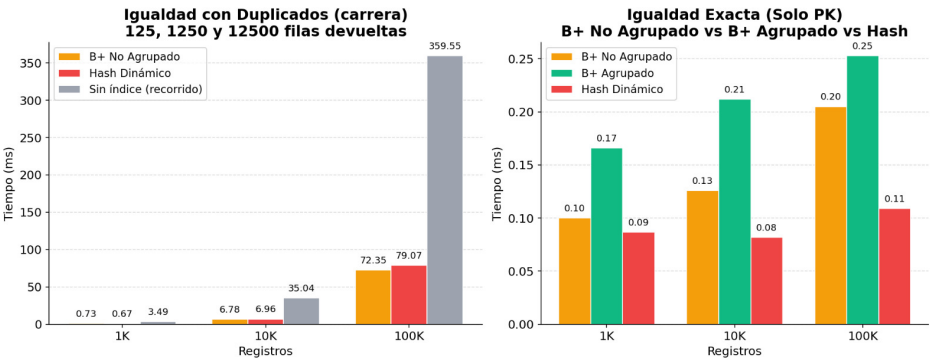


Figura 45. Búsqueda por igualdad sobre carrera, a la izquierda, y por clave primaria, a la derecha.

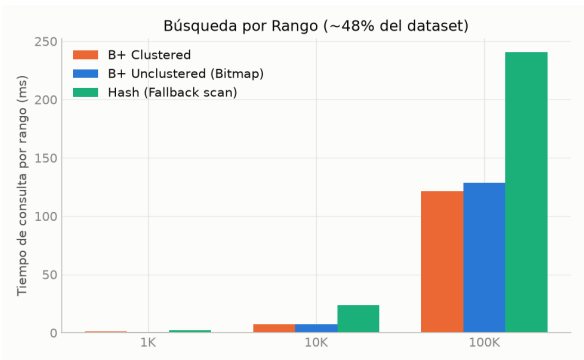


Figura 46. Búsqueda por rango sobre el 1 % y el 50 % de la tabla.

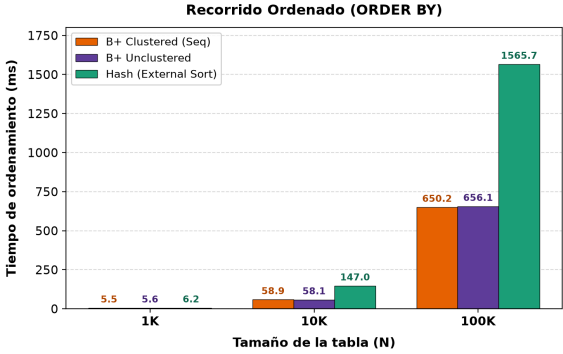


Figura 47. ORDER BY devolviendo todas las filas.

Construcción y espacio. El B+ agrupado es disperso y guarda una entrada por página de MAIN, 1 348 entradas para 100K filas, así que ocupa 28 KB y se construye en 261 ms. El B+ no agrupado y el hash tienen una entrada por fila. El B+ no agrupado ocupó 3 480 KB, unos 36 bytes por entrada, y el hash 1 587 KB. En el [cuadro 10](#) los dos índices densos se construyeron insertando una entrada a la vez, mientras que el agrupado se arma con una carga masiva sobre MAIN, así que la diferencia de tiempo mezcla los dos efectos. Con bulk load ([apartado 6.2](#)), el B+ no agrupado de 100K se construyó en 124 ms en la corrida de las consultas, frente a 28 s insertando de a una entrada.

Igualdad. Por clave primaria, las tres estructuras devuelven la fila en menos de 0,3 ms en todos los tamaños ([figura 45](#)). El hash es el más rápido (0,11 ms en 100K), porque calcula el bucket y lee la fila sin recorrer niveles, y su tiempo casi no cambia con *N*. El B+ no agrupado tarda 0,20 ms y crece levemente con la altura del árbol. El B+ agrupado tarda 0,25 ms aunque su árbol es más bajo, porque al llegar a la página de MAIN decodifica

sus registros uno por uno hasta encontrar la clave, unos 37 en promedio, mientras que el no agrupado lee directamente el slot que indica el RID. Sobre **carrera**, con 12 500 filas por valor en 100K, el hash (79,07 ms) y el B+ no agrupado (72,35 ms) tardan casi lo mismo, porque el costo lo domina leer y deserializar las filas devueltas, unos 6 μ s por fila, y los dos son unas 5 veces más rápidos que recorrer la tabla (359,55 ms).

Rango. Con un rango del 1 % de la tabla, los dos B+ devuelven las 1 000 filas en unos 5 ms con 100K, unas 70 veces menos que el hash, que no conserva el orden y tiene que recorrer la tabla (354 ms) (figura 46). El B+ agrupado lee páginas consecutivas de MAIN y el no agrupado lee sus filas con el bitmap, una vez por página. Con un rango del 50 % la ventaja casi desaparece (309 y 278 ms frente a 394 ms), porque de todas formas hay que leer y deserializar la mitad de la tabla. En esta prueba las filas se insertaron en orden de **id**, así que el Heap quedó ordenado por la clave y el bitmap del no agrupado lee páginas contiguas, como el agrupado. Con un Heap en orden de llegada aleatorio, el no agrupado leería páginas salteadas.

ORDER BY. Para devolver las 100 000 filas ordenadas por **id**, la tabla Sequential sigue su lista enlazada (656 ms) y el B+ no agrupado recorre sus hojas y lee cada fila por su RID (635 ms), así que tardan casi lo mismo (figura 47). El B+ agrupado no interviene, porque la lista ya entrega los registros en orden desde la cabeza. Sin un índice que dé el orden, como con el hash, hay que recorrer la tabla y ordenar con external sort, que con 64 páginas de buffer escribió 18 runs y tardó 1 555 ms, unas 2,4 veces más. En los tres casos la mayor parte del tiempo es deserializar las filas, así que tener las filas en orden ahorra el ordenamiento pero no la lectura.

11.3.1 Inserciones y eliminaciones intercaladas

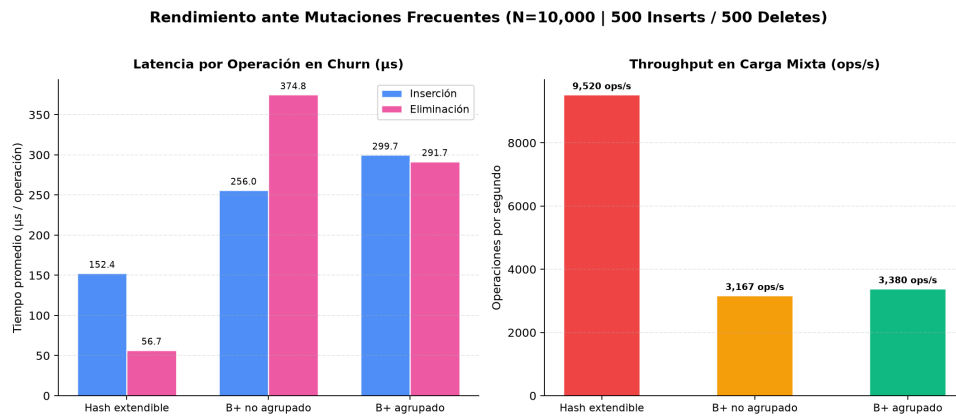
Para ver cómo se comportan las tres estructuras con inserciones y eliminaciones mezcladas (apartado 2.1.6 del enunciado), se partió de $N = 10\,000$ filas y se hicieron 1 000 operaciones intercaladas: 500 inserciones de claves intermedias y 500 eliminaciones de filas existentes elegidas al azar (cuadro 13 y figura 48). Las dos primeras filas del cuadro indexan tablas Heap y la tercera una tabla Sequential, así que la tercera incluye el costo de AUX, de la lista enlazada y de la política de reorganización, y no solo el del índice.

El experimento de mutaciones intensivas (cuadro 13 y figura 48) expone los compromisos arquitectónicos fundamentales entre estructuras dinámicas y basadas en orden físico:

1. **Supremacía del Hash extendible en cargas OLTP:** El índice Hash logra el mayor *throughput* global del sistema con **9 520 ops/s**, registrando una latencia promedio de inserción de 152,4 μ s y una eliminación ultrarrápida de 56,7 μ s. Dado que la localización de la página física se calcula mediante una dispersión $\mathcal{O}(1)$ sobre el directorio, las modificaciones se resuelven exclusivamente dentro de la página del *bucket* correspondiente sin desencadenar divisiones verticales ni actualizaciones en cascada.
2. **Respuesta del B+ agrupado sobre Sequential File:** La combinación de archivo secuencial y B+ agrupado mantiene un rendimiento sólido de **3 380 ops/s**, con una

Cuadro 13. Rendimiento de los índices relacionales ante mutaciones frecuentes ($N = 10\,000$, 500 inserciones y 500 eliminaciones intercaladas).

Índice	Inserción (μs)	Eliminación (μs)	Throughput (ops/s)	Tamaño (KB)	Impacto físico final
Hash extendible	152,4	56,7	9 520	264,0	Acceso $\mathcal{O}(1)$ directo en <i>bucket</i> ; eliminación lógica sin rebalanceo estructural.
B+ agrupado (Seq)	299,7	291,7	3 380	8,0	Desbordes encadenados en AUX (7 págs.); 4,8 % borrados; reorganización requerida.
B+ no agrupado	256,0	374,8	3 167	360,0	Costo $\mathcal{O}(\log_B N)$; balanceo por división de nodos y mantenimiento de punteros RID.

**Figura 48.** Tiempo por operación y operaciones por segundo con inserciones y eliminaciones intercaladas.

latencia equilibrada entre inserciones ($299,7 \mu s$) y eliminaciones ($291,7 \mu s$). La estructura absorbe eficientemente las claves intermedias fuera de orden derivándolas hacia el área auxiliar encadenada (AUX) mediante punteros lógicos, evitando desplazamientos masivos en las páginas MAIN. Sin embargo, este mecanismo traslada el costo hacia la fragmentación física: al concluir las 1 000 operaciones, el archivo acumuló 7 páginas en AUX y un 4,8 % de espacio ocupado por registros eliminados lógicamente, alcanzando el umbral que dispara la reorganización periódica fuera de línea.

3. Estabilidad predecible del B+ no agrupado: Con un rendimiento de 3 167

ops/s, el árbol B+ no agrupado demuestra una alta regularidad logarítmica. La eliminación resulta ligeramente más costosa ($374,8 \mu\text{s}$) que la inserción ($256,0 \mu\text{s}$) debido a las operaciones de verificación de subflujo (*underflow*) y a las rutinas de redistribución o fusión de nodos hoja necesarias para mantener balanceado el árbol denso.

11.4 Búsqueda secuencial, R-Tree y GiST de PostgreSQL

Se esperaba que la búsqueda secuencial creciera linealmente con N , que el R-Tree y GiST descartaran la mayor parte de los puntos en consultas pequeñas, y que el R-Tree perdiera esa ventaja en radios grandes, donde entra buena parte de la tabla. En k-NN, el recorrido best-first descarta ramas por MINDIST sin ordenar todos los puntos ([apartado 6.5](#)).

La búsqueda secuencial calcula la distancia a los 100K puntos sin índice: no usa disco adicional ni tiempo de construcción, y su memoria es la que ocupan los puntos en Python ([cuadro 14](#)). Cuesta unos $0,7 \mu\text{s}$ por punto (unos 71 ms para 100 000 puntos), menos que deserializar una fila con `Record.unpack`, así que la comparación favorece a la búsqueda secuencial frente al R-Tree, que sí lee páginas a través del buffer pool.

PostGIS usa SRID 4326, con coordenadas (*lon, lat*) y el tipo `geography`, y el índice GiST se crea sobre esa representación. Las consultas que se midieron son las del [listado 5](#).

```
1 CREATE INDEX idx_tiendas_gist
2 ON tiendas USING GIST ((geom::geography));
3
4 -- Radio en metros sobre geography.
5 SELECT id, geom
6 FROM tiendas
7 WHERE ST_DWithin(
8     geom::geography,
9     ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::geography,
10    :radio_m
11 );
12
13 -- k-NN con distancia esferica sobre geography.
14 SELECT id, geom
15 FROM tiendas
16 ORDER BY geom::geography <-> ST_SetSRID(ST_MakePoint(:lon, :lat), 4326)::
17     geography
18 LIMIT :k;
```

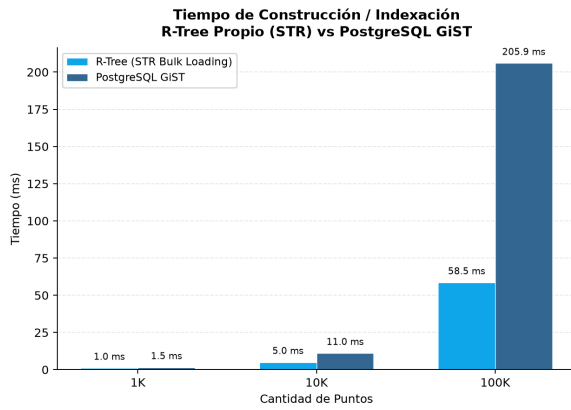
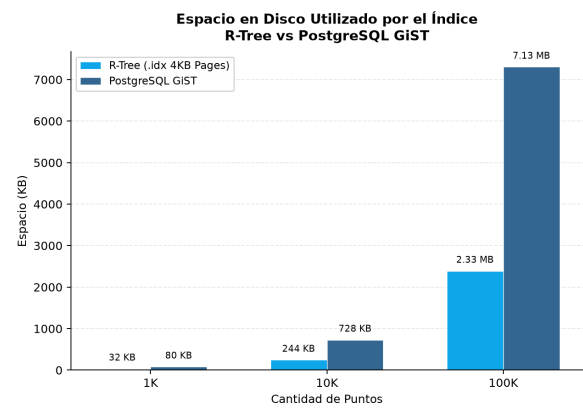
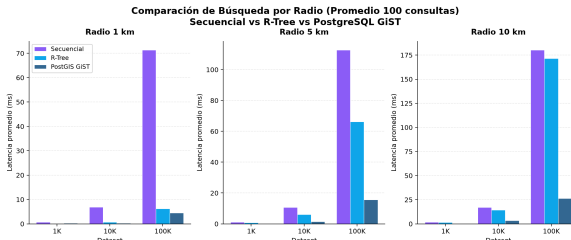
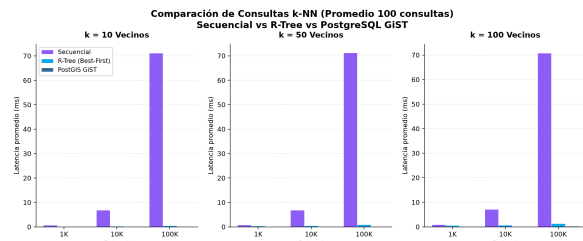
Listing 5. Consultas equivalentes en PostGIS.

La comparación tiene cuatro sesgos conocidos: la caché caliente, los 1 MB del buffer pool frente a los 128 MB de `shared_buffers`, el costo del conector `psycpg2` y la diferencia entre medir objetos de Python y páginas en disco. Para reducirlos se midió solo el motor, sin red ni interfaz, y se documentaron todos los parámetros. La diferencia de caché no se corrigió.

Cuadro 14. Construcción, espacio en disco y memoria de las técnicas espaciales.

N	Secuencial			R-Tree propio			PostGIS GiST		
	ms	Disco	RAM	ms	Disco	RAM	ms	Disco	RAM
1 000	0,00	0,0 KB	71,1 KB	1,04	32,0 KB	≤ 32 KB*	1,47	80,0 KB	Buffer [§]
10 000	0,00	0,0 KB	708,2 KB	4,96	244,0 KB	≤ 244 KB*	11,02	728,0 KB	Buffer [§]
100 000	0,00	0,0 KB	7 032,2 KB [†]	58,52	2 388,0 KB	≤ 1024 KB [†]	206,92	7 304,0 KB	Buffer [§]

*Hasta 10K el índice entero cabe en el buffer pool. [†]En 100K las páginas del índice en memoria están acotadas por el buffer pool (256 frames de 4 KB); la cota no incluye las filas del resultado. [‡]Memoria de los objetos de Python (`sys.getsizeof`). [§]GiST usa el `shared_buffers` de PostgreSQL (128 MB); su memoria no se midió por separado.

**Figura 49.** Tiempo de construcción del R-Tree y de GiST.**Figura 50.** Espacio en disco del R-Tree y de GiST.**Figura 51.** Consultas por radio (1, 5 y 10 km).**Figura 52.** Consultas k-NN ($k = 10, 50, 100$).

De los cuadros 14 y 15 se observa lo siguiente.

k-NN. Con 100K puntos y $k = 10$ (figura 52), la búsqueda secuencial tarda 71,23 ms, casi lo mismo para cualquier k , porque siempre calcula la distancia a todos los puntos. El R-Tree tarda 0,52 ms, unas 137 veces menos, porque el recorrido best-first deja sin leer las ramas lejanas. GiST tarda 0,16 ms.

Radio de 1 km. Esta consulta devuelve cerca del 1 % de la tabla (figura 51). El R-Tree tarda 6,25 ms y GiST 4,46 ms; según EXPLAIN, PostgreSQL usó Bitmap Index Scan on `idx_tiendas_gist`.

Cuadro 15. Tiempo promedio de 100 consultas espaciales en ms.

N	Consulta	Secuencial	R-Tree	GiST
1 000	Radio 1 km	0,70	0,16	0,28
	Radio 5 km	1,08	0,75	0,23
	Radio 10 km	1,68	1,59	0,36
	k-NN $k = 10$	0,71	0,20	0,06
	k-NN $k = 50$	0,81	0,38	0,09
	k-NN $k = 100$	0,93	0,61	0,13
10 000	Radio 1 km	6,89	0,71	0,29
	Radio 5 km	10,70	6,05	1,39
	Radio 10 km	16,99	14,47	3,39
	k-NN $k = 10$	6,84	0,25	0,12
	k-NN $k = 50$	6,90	0,49	0,15
	k-NN $k = 100$	7,09	0,74	0,17
100 000	Radio 1 km	71,40	6,25	4,46
	Radio 5 km	112,81	66,23	15,61
	Radio 10 km	180,37	171,54	26,26
	k-NN $k = 10$	71,23	0,52	0,16
	k-NN $k = 50$	71,19	0,94	0,21
	k-NN $k = 100$	70,87	1,36	0,27

Radio de 10 km. Un círculo de 20 km de diámetro en Lima contiene entre el 45 y el 48 % de las tiendas, unas 45 800 filas. El R-Tree casi no puede descartar ramas y termina leyendo casi todas las páginas del índice y de la tabla, así que su tiempo (171,54 ms) se acerca al de la búsqueda secuencial (180,37 ms). Ese tiempo es sobre todo deserializar las filas devueltas, unos $3,7 \mu s$ por fila. GiST devuelve lo mismo en 26,26 ms, unas 6,9 veces menos que la búsqueda secuencial, lo que indica que el límite del R-Tree propio está en el costo por fila de Python y no en la estructura.

R-Tree frente a GiST. GiST fue más rápido en 17 de las 18 configuraciones; el R-Tree solo ganó en 1K con radio de 1 km (0,16 frente a 0,28 ms). En cambio, el R-Tree se construyó unas 3,5 veces más rápido (58,52 frente a 206,92 ms en 100K) y ocupó menos de un tercio del disco (2 388 frente a 7 304 KB) (figura 50). Sus hojas guardan entradas fijas de 24 bytes en páginas de 4 KB (apartado 6.5), mientras que GiST usa páginas de PostgreSQL con cabeceras de tupla de 24 bytes.

11.5 Cuándo usar cada técnica

Cuadro 16. Consultas que resuelve cada índice relacional.

Índice	Igualdad	Rango	Orden	Observación
B+ no agrupado	Sí	Sí	Sí	Lee las filas con el bitmap, una vez por página; hojas densas.
B+ agrupado	Sí	Sí	Sí	Disperso sobre MAIN; siempre recorre AUX.
Hash extendible	Sí	No	No	El más rápido en igualdad; con muchos duplicados cuesta lo mismo que un B+, porque domina leer las filas.

Cuadro 17. Comparación de los índices relacionales.

Índice	A favor	En contra	Cuándo conviene
B+ no agrupado	Igualdad, rango y orden sobre cualquier columna; en rango del 1 % lee las filas en 5 ms con 100K; 3 167 ops/s con inserciones y eliminaciones intercaladas.	3,5 MB para 100K (unos 36 bytes por entrada); construirlo insertando una entrada a la vez tardó 18,15 s, y con bulk load unas 230 veces menos.	Columnas que no son la clave, con consultas de igualdad y de rango y escrituras frecuentes.
B+ agrupado	Muy pequeño (28 KB para 100K) y rápido de construir (261 ms); en rango lee páginas consecutivas de MAIN sin pasar por RIDs.	Solo sobre la clave de un Sequential File; en igualdad es algo más lento que el no agrupado (0,25 frente a 0,20 ms), porque decodifica la página de MAIN; las inserciones intermedias van a AUX (0,30 ms por inserción, 3 380 ops/s) y obligan a reorganizar.	Tablas que cambian poco o reciben claves en orden, con consultas por rango sobre la clave.
Hash extendible	El más rápido en igualdad por clave (0,11 ms en 100K) y con inserciones y eliminaciones (9 520 ops/s, 56,7 μ s por eliminación).	No resuelve rango ni orden (354 y 1 555 ms en 100K, recorriendo la tabla y ordenando con external sort); no admite FLOAT ni DOUBLE.	Igualdad sobre cualquier columna sin consultas por rango ni orden, con escrituras frecuentes.

Cuadro 18. Comparación de las organizaciones de archivo.

Organización A favor		En contra	Cuándo conviene
Heap File	Inserción de 2 I/O si cabe en la última página ($\mathcal{O}(c)$ si no); no necesita reorganizar; reutiliza slots libres; 29,5 % menos espacio que el Sequential (cuadro 8).	Sin índice, buscar exige recorrer el archivo, $\mathcal{O}(B)$; no mantiene orden; en la carga ascendente de 100K fue más lento que el Sequential (5,16 frente a 3,99 s).	Tablas con muchas inserciones y actualizaciones que se consultan por índices.
Sequential File	Registros ordenados por clave; búsqueda por clave en $\mathcal{O}(\log B)$ (0,15 ms en 100K); rangos y recorridos ordenados sin ordenar.	Las claves intermedias van a AUX (0,30 ms por inserción, apartado 11.3.1); reorganizar cuesta $\mathcal{O}(B + A)$ (2,31 s en 100K); ocupa más espacio.	Tablas que reciben claves en orden y se consultan por clave o por rango de clave.

Cuadro 19. Comparación de las técnicas espaciales.

Técnica	A favor	En contra	Cuándo conviene
Búsqueda secuencial	No usa disco adicional ni se construye; es la más simple.	Crece linealmente con N : 71,4 ms en un radio de 1 km con 100K, frente a 6,25 ms del R-Tree.	Tablas pequeñas (con 1K todas las consultas tardan menos de 2 ms) o sin consultas espaciales frecuentes.
R-Tree propio	k-NN en menos de 1,4 ms con $k \leq 100$ en 100K; se construye 3,5 veces más rápido que GiST y ocupa un tercio del disco; usa el mismo buffer pool que el resto del motor.	En radios grandes (10 km, cerca del 45 % de la tabla) tarda lo mismo que la búsqueda secuencial; más lento que GiST en 17 de 18 casos; con muchos borrados sus nodos quedan poco llenos y conviene reconstruirlo con STR (apartado 6.5).	Consultas de cercanía y k-NN con k pequeño dentro del propio motor.
PostGIS (GiST)	Geometrías complejas y un optimizador por costos; 6,5 veces más rápido que el R-Tree propio con radio de 10 km en 100K.	Ocupa más de 3 veces el disco del R-Tree (7 304 frente a 2 388 KB); se construye más lento; necesita un servidor PostgreSQL.	Sistemas grandes con joins espaciales, transacciones concurrentes y polígonos arbitrarios.

12. Conclusiones

El sistema persiste páginas y registros, mantiene dos organizaciones de archivo y cuatro índices, ejecuta algoritmos externos, interpreta SQL, maneja transacciones concurrentes y muestra el plan de cada consulta. Tener Heap y Sequential junto con B+ y hash permite medir en el mismo motor cuánto cuesta cada decisión en carga, orden, espacio y búsqueda.

Todas las estructuras comparten la misma pila de almacenamiento: páginas de 4 KB que lee y escribe el FileManager, un buffer pool que escribe al momento y una free list por archivo de índice. Eso simplificó el rollback, que lee la versión anterior de cualquier página desde el archivo, y permitió que el inspector físico muestre las páginas reales de cada índice.

La parte espacial usa el mismo catálogo, SQL, transacciones e interfaz que la relacional. El R-Tree trata los puntos como MBR degenerados, se construye con STR y resuelve radio, polígono y k-NN con MINDIST y MAXDIST en las dos métricas. Como el recorrido completo evalúa la misma función que el índice, cada consulta con índice se puede comparar con su resultado secuencial.

Las limitaciones principales son de alcance. El planificador aplica reglas sin estadísticas,

los locks son por tabla, el undo existe solo en memoria y no hay recuperación ante caídas, y el SQL cubre lo necesario para usar las estructuras implementadas. En los experimentos, los cuadros relacionales usan datos distintos entre sí y algunas mediciones son de una sola ejecución, así que las conclusiones siguientes se apoyan en tendencias dentro de cada cuadro y no en comparar cifras entre cuadros. De los experimentos se concluye lo siguiente.

1. **Heap y Sequential.** El Heap File insertó más rápido con 1K y 10K filas, ocupó un 29,5 % menos y no necesita reorganizar, pero buscar sin índice obliga a recorrerlo (133,8ms en 100K). Con 100K filas en orden ascendente insertó más lento que el Sequential (5,16 frente a 3,99 s), porque el archivo ya no cabe en el buffer pool (2,4 MB frente a 1 MB) y el mapa de espacio libre crece. El Sequential File busca por clave en 0,15 ms con su B+ agrupado, a cambio de más espacio, inserciones intermedias en AUX (0,30 ms) y reorganizaciones (2,31 s en 100K).
2. **Índices relacionales.** Ninguno es mejor en todo. El hash fue el más rápido en igualdad por clave (0,11 ms en 100K) y con inserciones y eliminaciones (9 520 ops/s), pero no resuelve rango ni orden, y sin un índice que dé el orden, el ORDER BY con external sort tardó unas 2,4 veces más (1 555 frente a 656 ms). Con muchos duplicados, el hash y el B+ no agrupado cuestan lo mismo, porque domina leer las filas. En rangos pequeños los dos B+ son unas 70 veces más rápidos que recorrer la tabla, y en rangos de la mitad de la tabla la ventaja casi desaparece. El B+ agrupado es el más pequeño (28 KB para 100K) y el más rápido de construir (261 ms), y la tabla Sequential que lo usa alcanzó 3 380 ops/s con inserciones intercaladas. El B+ no agrupado ocupa más (3,5 MB) y construido de a una entrada tardó 18,15 s, frente a unas 230 veces menos con bulk load.
3. **Índices espaciales.** La búsqueda secuencial solo es aceptable con tablas pequeñas: con 100K tarda 71,4ms en un radio de 1 km. El R-Tree construido con STR tardó 58,52 ms y ocupó 2,33 MB, unas 3,5 veces menos tiempo que GiST (206,92 ms y 7,13 MB). Fue 11,4 veces más rápido que la búsqueda secuencial en un radio de 1 km (6,25 frente a 71,40 ms) y unas 137 veces en k-NN con $k = 10$ (0,52 frente a 71,23 ms). Con un radio de 10 km, donde entra cerca del 45 % de la tabla, tarda casi lo mismo que la búsqueda secuencial (171,5 frente a 180,4 ms) por el costo de deserializar cada fila en Python, mientras que GiST tarda 26,3 ms. GiST fue más rápido en 17 de las 18 configuraciones.
4. **Memoria.** El buffer pool limita a 1 MB las páginas que el motor guarda en caché, y eso le permite recorrer tablas e índices más grandes, como el R-Tree de 100K (2,33 MB). No limita la memoria total del proceso, que también incluye el directorio del hash, el mapa de espacio libre del Heap, los 256 KB de los algoritmos externos y las filas de cada resultado. El único dato de memoria medido es el de los puntos de la búsqueda secuencial (unos 7 MB con 100K); no se midió el consumo real del proceso.

En conclusión, la estructura adecuada depende de la carga: lo que una estructura gana en las consultas lo paga en mantenimiento al escribir y en espacio. Los experimentos mostraron cuánto pesa el buffer pool cuando el archivo no cabe en él, que el R-Tree

propio compite con PostGIS en consultas espaciales pequeñas, y que la reorganización conviene medirla por separado de las inserciones.

Referencias

- [1] Universidad de Ingeniería y Tecnología, *01 Arquitectura de SGBD*, material del curso Base de Datos II, 2026.
- [2] Universidad de Ingeniería y Tecnología, *02 Registros, paginación y organización de archivos*, material del curso Base de Datos II, 2026.
- [3] Universidad de Ingeniería y Tecnología, *03 Indexación Hash y Árboles B*, material del curso Base de Datos II, 2026.
- [4] Universidad de Ingeniería y Tecnología, *04 Bitmap Index Scan y algoritmos externos*, material del curso Base de Datos II, 2026.
- [5] Universidad de Ingeniería y Tecnología, *05 Recuperación ante fallos*, material del curso Base de Datos II, 2026.
- [6] Universidad de Ingeniería y Tecnología, *07 Bases de datos espaciales: GiST y R-Tree*, material del curso Base de Datos II, 2026.
- [7] R. Ramakrishnan y J. Gehrke, *Database Management Systems*, 3.^a ed., McGraw-Hill, 2003.
- [8] R. Fagin, J. Nievergelt, N. Pippenger y H. R. Strong, “Extendible Hashing: A Fast Access Method for Dynamic Files”, *ACM Transactions on Database Systems*, vol. 4, núm. 3, pp. 315–344, 1979.
- [9] A. Guttman, “R-Trees: A Dynamic Index Structure for Spatial Searching”, *Proceedings of ACM SIGMOD*, pp. 47–57, 1984.
- [10] S. T. Leutenegger, M. A. López y J. Edgington, “STR: A Simple and Efficient Algorithm for R-Tree Packing”, *Proceedings of the 13th International Conference on Data Engineering (ICDE)*, pp. 497–506, 1997.

Anexos

A continuación se muestran los enlaces al código fuente y página web desplegada:

- **Repositorio del proyecto:**
<https://github.com/iranalvarez27/Proyecto-BD2>
- **Web:**
<https://proyecto-bd2.vercel.app/>